Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation
Software Engineering Study Programme

PBL REPORT

STUDENT FORUM

Team No: 15

Mentors: Cazac Marin
Cara Alexandru

Team members: Belih Dmitrii, FAF-232
Chicu Andrei, FAF-233
Postoronca Dumitru, FAF-233
Racovita Dumitru, FAF-233
Titerez Vladislav, FAF-233

Chisinau, 2025

ABSTRACT

The thesis titled **Student Forum Platform: Enhancing Academic Feedback and Communication in Higher Education** was developed by student *LastName FirstName* as a bachelor's project at the *Technical University of Moldova*. It is written in English and contains an introduction, 4 chapters (Domain Analysis, Requirements Specification, System Design, System Implementation), a list of figures, tables, conclusion, bibliography, and appendices.

In this study, we address the **problem of limited and inefficient student feedback systems** in higher education, where communication between students and institutions remains infrequent, fragmented, and often anonymous only in form. The research identifies the shortcomings of existing solutions such as Moldova's SIMU platform, which collects evaluations only once per semester and provides no open, anonymous peer discussion channel.

To solve this, the proposed **Student Forum Platform** introduces a secure, centralized, and anonymous environment enabling students to exchange opinions, post course feedback, and participate in polls. The system is built using **Next.js**, **Supabase**, and **Drizzle ORM**, ensuring scalability, performance, and type-safe database operations. Authentication and data protection are achieved through **OAuth 2.0**, **Multi-Factor Authentication (MFA)**, and **JWT-based session handling**, complying with modern web security standards.

Initial testing and comparative analysis show improved engagement and transparency compared to traditional feedback systems. By integrating anonymity, verified access, and community-driven features, the platform fosters a culture of open communication in academic environments. The project's anticipated impact is a **more responsive, student-centered education ecosystem**, where real-time feedback contributes to continuous institutional improvement.

Keywords: Student forum, feedback platform, academic communication, Supabase, Next.js, Drizzle ORM, OAuth2, MFA, higher education

TABLE OF CONTENTS

INTRODUCTION

Changes in technology continue to alter the possibilities for learning and create new challenges for pedagogy. Over the last two decades, colleges and universities have adapted and responded to innovations such as the Internet, email, instant messaging, learning management systems, podcasts, and mobile applications. The rapid evolution of mobile and web technologies has transformed how students and educators interact, collaborate, and access information. Among learners aged 12–25, the Internet has become not only a primary source of knowledge but also a social environment for communication and exchange of ideas.

In this context, the integration of web technologies into higher education represents both an opportunity and a necessity. Modern students expect accessible, interactive, and real-time platforms that support engagement beyond traditional classroom settings. However, many existing academic tools remain limited — they often focus only on administrative or assessment functions and lack open spaces for discussion, peer feedback, and community building. The absence of such platforms restricts authentic communication, hinders collaborative learning, and isolates students within fragmented digital ecosystems.

This project aims to address these challenges by developing a web-based forum application designed to facilitate communication between students and teachers, as well as among peers. The platform allows users to exchange information on academic and non-academic topics, from coursework to personal interests, and to join thematic channels to discuss ideas, share resources, and find solutions to common problems. By fostering open, topic-driven communication, the system promotes inclusivity, collaboration, and knowledge sharing in a secure online environment.

The motivation for this project stems from the growing demand for platforms that bridge the gap between social media interaction and academic collaboration. While tools like Facebook or Discord offer communication features, they lack academic focus and structured discussion spaces. Conversely, university feedback systems tend to be formal, infrequent, and non-interactive. The proposed platform combines the strengths of both approaches—maintaining academic relevance while ensuring accessibility and engagement.

The potential impact of this solution lies in its ability to enhance student participation, improve the flow of academic feedback, and encourage continuous learning. It empowers students to express their opinions anonymously when needed, enabling honest feedback and fostering a more transparent educational culture.

1 DOMAIN ANALYSIS

Leak of centralize platform which gives student opportunity to post their thought and question, coupled with the limitations of the end-of-semester SIMU feedback system, restricts timely academic improvements and leaves prospective students with insufficient authentic information about university life.

1.1 Problem Definition

In Moldova, there is a significant gap in platforms that enable students to communicate, share knowledge, and exchange experiences, unlike in other countries where such platforms are common. The primary mechanism for student feedback, the end-of-semester evaluation platform (SIMU), is limited to collecting feedback once per semester without facilitating interaction or discussion among students. This restricts timely improvements to the academic experience. Additionally, students perceive a lack of anonymity in SIMU, fearing academic reprisal, which discourages honest and constructive feedback. Furthermore, prospective students rely on curated university marketing materials and anecdotal stories, which fail to provide authentic insights into daily student life, leaving critical questions about courses and campus experiences unanswered. A solution is needed to create a platform that fosters continuous, anonymous, and transparent communication and knowledge-sharing among students, enhancing the academic environment and providing reliable information for prospective students.

- **Infrequent Feedback Collection**: The SIMU platform collects feedback only once per semester, at its conclusion. This timing prevents instructors and administrators from implementing timely changes to address student concerns, rendering the feedback ineffective for improving the ongoing academic experience.
- **Perceived Lack of Anonymity**: Despite assurances of anonymity, many students fear that their critical feedback could be traced back to them. This apprehension, stemming from concerns about potential academic reprisal from professors, discourages honest and constructive criticism, undermining the quality and candor of the feedback provided.
- **Limited Information for Prospective Students**: Their primary sources are official marketing materials and anecdotal stories. While helpful, marketing content is designed to present the university in the best possible light and often fails to capture the day-to-day realities of student life. This leaves critical questions unanswered about course difficulty, professor teaching styles, campus culture, and the true student routine.

To address these challenges, our proposed solution, the Student Forum application, aims to provide a platform that facilitates continuous, anonymous, and transparent feedback from students. This platform will

empower students to share honest insights and experiences in real time, fostering a more responsive academic environment and providing prospective students with a clearer, more authentic picture of university life.

1.2 Problem Analysis

To better understand [1] the challenges students face and to validate the need for a dedicated communication and feedback platform, we conducted a survey with university students across different years of study. The survey explored frustrations during the semester, feedback practices, access to information, and communication habits.

Frustrations During Semester:

Students most frequently reported difficulties with managing schedules and deadlines, finding reliable study partners, and organizing group projects. These frustrations highlight the need for a structured tool that simplifies coordination and improves access to peer support.

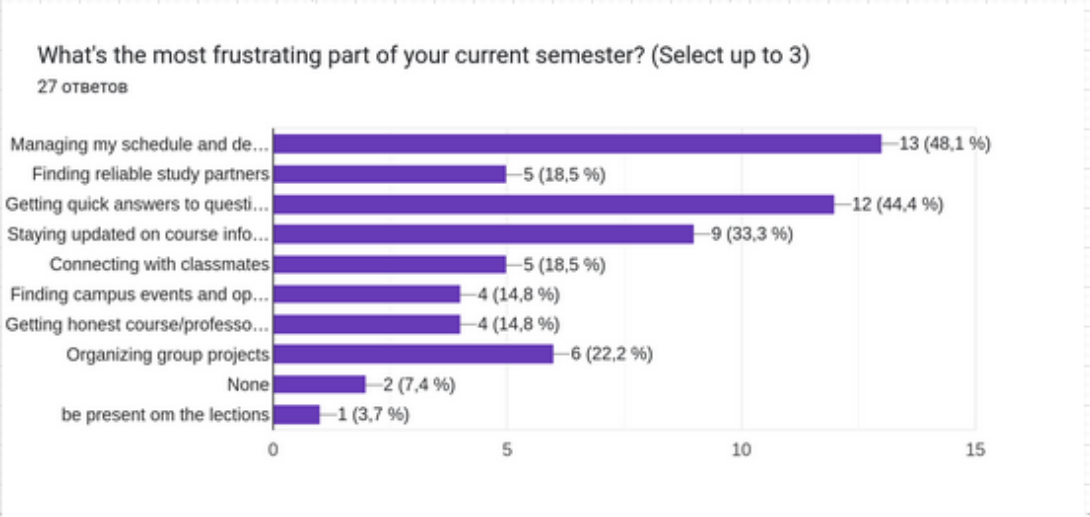


Figure 1.1 - Most Frustrating Aspects of Current Semester

As shown in Figure 1.1, managing schedules and deadlines (48.1%) emerged as the primary concern, followed by finding reliable study partners (18.5%) and getting quick answers to questions (44.4%). These statistics reinforce the need for a centralized platform that addresses these specific pain points. [2]

Importance of Knowing Professors:

On a scale from 1 (not important) to 5 (very important), students rated an average of around 4, indicating that knowledge about professors' teaching styles significantly influences their academic choices. However, most information is currently gathered informally through word of mouth or social media, showing the lack of an official and transparent source.

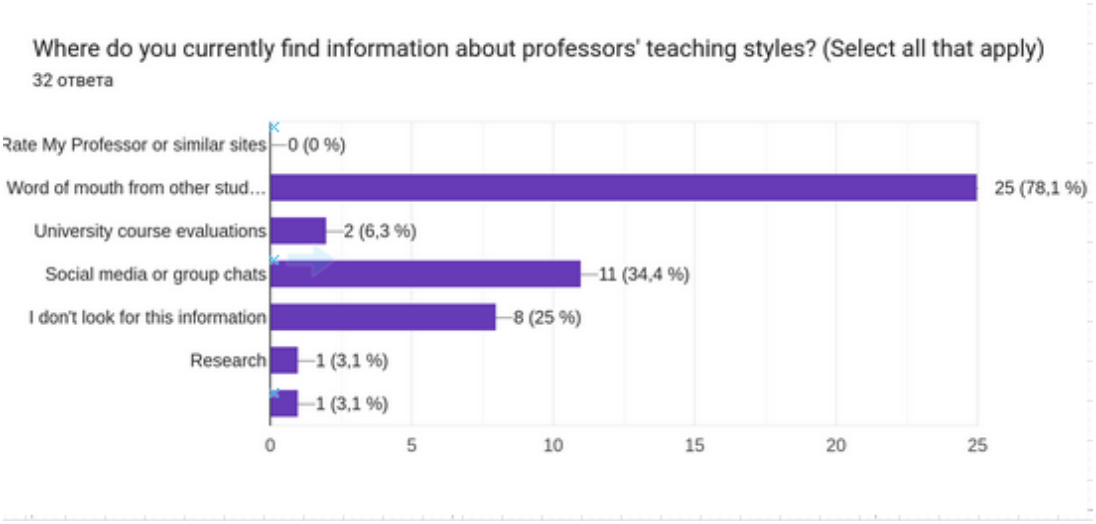


Figure 1.2 - Sources of Information About Professors' Teaching Styles

As shown in Figure 1.2, word of mouth from other students (78.1%) is the predominant source of information about professors, followed by social media or group chats (34.4%). Notably, only 6.3% of students use official university course evaluations, highlighting the need for a more formalized and accessible platform for sharing this information.

Barriers to Asking Questions:

A large portion of respondents cited fear of seeming unprepared, discomfort in speaking up, or not wanting to waste others' time as reasons for avoiding questions in class or office hours. This demonstrates the need for anonymous channels of communication where students can seek help without judgment.

Finding Academic Support:

Students typically identify helpful classmates by observing participation in class or asking around randomly. This inefficient process shows potential for a platform that makes peer expertise and study partnerships more visible and accessible.

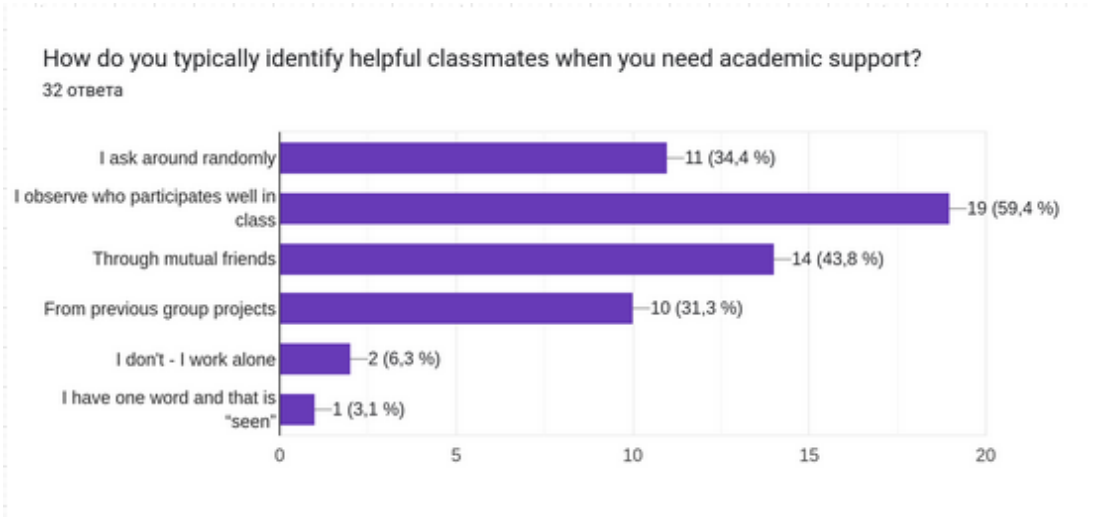


Figure 1.3 - Survey Results: Methods of Identifying Academic Support Partners

As shown in Figure 1.3, classroom participation observation (59.4%) is the primary method students use to identify potential study partners, followed by connecting through mutual friends (43.8%) and random asking (34.4%). Previous group project experience (31.3%) also plays a significant role. Only 6.3% of students prefer to work alone, indicating a strong need for collaborative learning support.

Communication and Coordination:

Most students rely on messaging apps and word of mouth to communicate with peers, but these are fragmented and lack academic focus. Additionally, students reported frequently missing important announcements or struggling to coordinate decisions like meeting times, further reinforcing the need for a centralized forum.

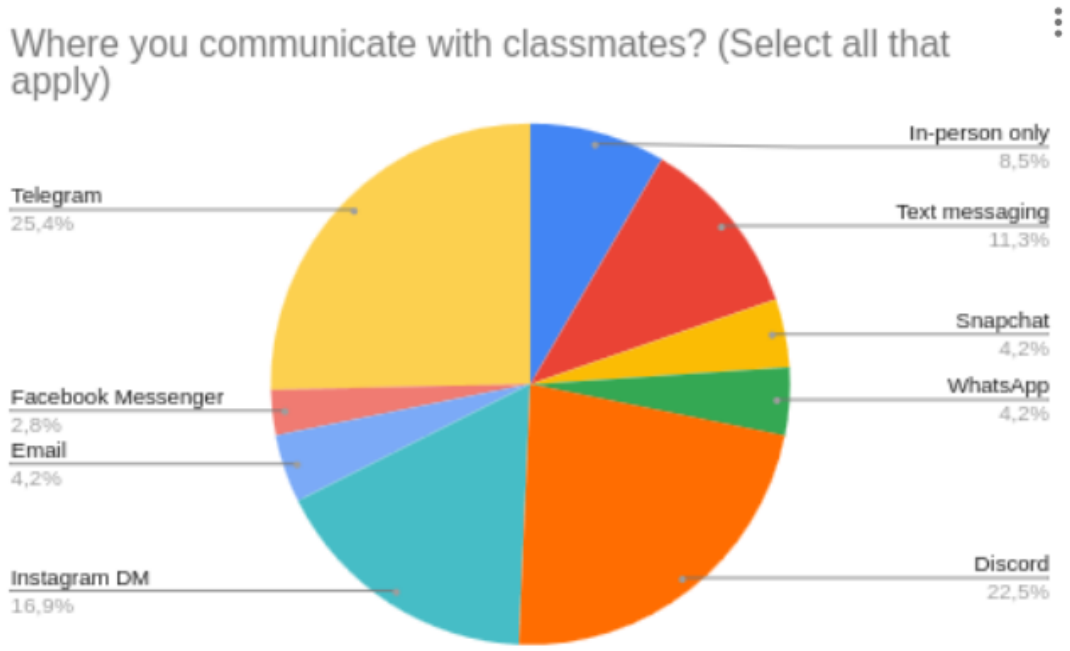


Figure 1.4 - Primary Communication Channels Among Students

As shown in Figure, Discord emerged as the primary communication platform, followed closely by Facebook Messenger and text messaging. Email remains relevant for formal communications, while platforms like Telegram and Instagram DMs are used by a smaller percentage of students. Notably, in-person communication accounts for only about 11.3% of interactions, highlighting the strong preference for digital communication methods among modern students.

Satisfaction with Current Tools:

When asked to rate satisfaction with existing tools on a scale from 1 to 10, the average rating was low to moderate, signaling clear room for improvement in digital platforms tailored to student needs.

Campus Events and Study Groups Discovery:

The survey investigated how students discover and stay informed about campus activities and study opportunities. Results show a diverse mix of both digital and traditional information channels.

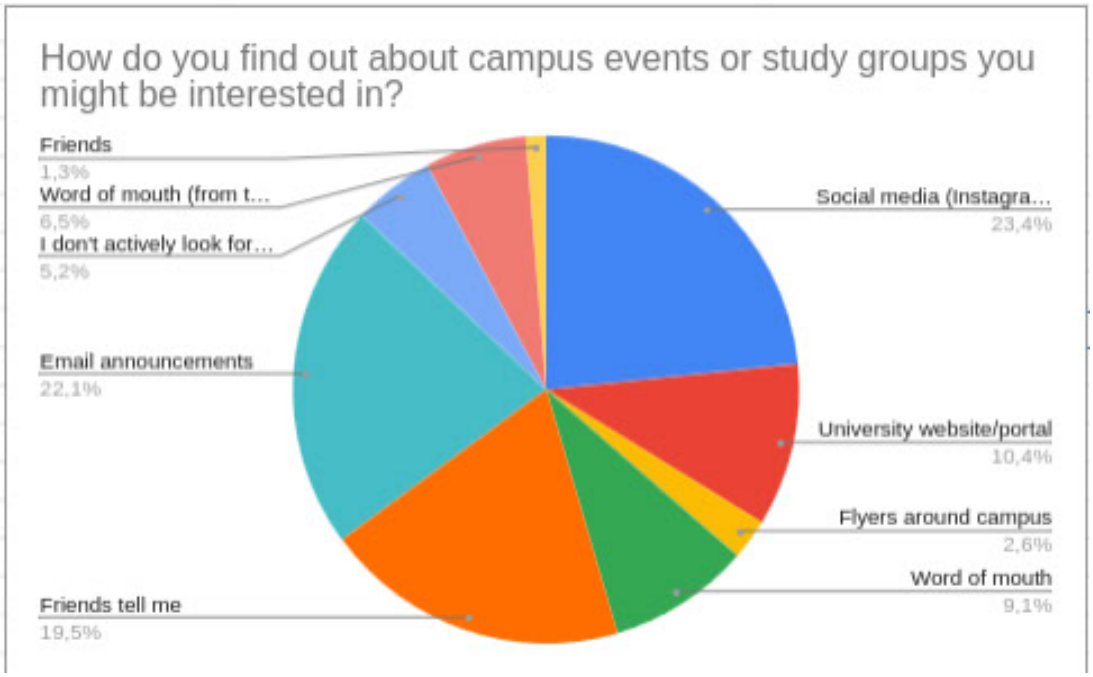


Figure 1.5 - Survey Results: Sources for Campus Events and Study Groups Information

Social media platforms (23.4%) represent the primary source for discovering campus events and study groups, followed closely by the university website/portal (22.1%). Email announcements (22.1%) maintain significant relevance, while traditional methods like flyers around campus (9.1%) and word of mouth (9.1%) still play a role. Notably, 5.2% of students report not actively seeking such information, suggesting a potential gap in engagement that could be addressed through a more centralized and accessible platform.

The survey confirms that students face recurring issues with communication, feedback, and access to reliable academic information. Current solutions are fragmented, informal, and often discourage participation due to lack of anonymity. A dedicated platform would address these gaps by enabling continuous, transparent, and anonymous communication, ultimately improving the academic environment and empowering students with authentic insights.

1.3 Solution Proposal

Based on the survey results and identified challenges, we propose the development of a **Student Forum Platform**. This solution directly addresses the needs of both prospective and currently enrolled [3] students by providing a centralized, anonymous, and transparent space for communication, feedback, and knowledge-sharing. The following arguments illustrate the necessity and impact of this platform.

1.3.1 Argument I: A Platform for Prospective Students

The Problem. Prospective students are often forced to make one of the most important decisions of their lives—choosing a university—based on limited and idealized sources of information. Their primary references are official marketing materials and anecdotal stories. While helpful, these materials are curated to present the university in the best light and rarely capture the realities of daily student life. As a result, important questions regarding course difficulty, professors' teaching styles, campus culture, and the overall student experience remain unanswered.

The Consequence. This lack of authentic information creates uncertainty and hesitation. Prospective students who cannot find genuine, peer-to-peer answers to their concerns may feel less confident in their choice and may opt for other institutions that appear more transparent and relatable.

Our Solution. The forum will offer a dedicated, searchable space where prospective students can directly ask questions to the university community. Current students can share their genuine experiences, providing an unfiltered perspective on courses, professors, and campus life. This peer-to-peer interaction creates a trusted and transparent resource, empowering prospective students to make more informed decisions and increasing the likelihood of their enrollment.

1.3.2 Argument II: A Platform Where Enrolled Students Are Heard

The Problem. For enrolled students, satisfaction and belonging are critical to their academic success and retention. However, the existing feedback system is inadequate. The current mechanism, the SIMU platform, suffers from two major limitations: Feedback is collected only once at the end of the semester, when it is too late to implement meaningful improvements. Despite assurances, many students believe their feedback is not truly anonymous. This fear of academic reprisal discourages open and constructive criticism.

The Consequence. This flawed feedback system creates an environment where students feel powerless and unheard. Unresolved concerns about teaching quality, course design, and administrative practices accumulate over time, leading to frustration, disengagement, and in some cases, student attrition.

Our Solution. The proposed forum provides a continuous, community-driven dialogue space where students can openly discuss academic and administrative issues in real time. Anonymous participation ensures that students feel safe sharing candid feedback without fear of reprisal. Beyond individual expression, the platform fosters collaborative problem-solving, allowing the student body to collectively identify issues and propose solutions. This cultivates a stronger sense of agency, involvement, and ownership in the academic community.

By addressing the needs of both prospective and current students, the Student Forum platform cre-

ates a dual impact: it strengthens the university's reputation and attractiveness for future applicants while simultaneously enhancing the academic environment for those already enrolled. This solution fosters transparency, trust, and collaboration, ultimately contributing to a more supportive and dynamic university community.

Table 1.1 - SWOT Analysis

Strengths	**Weaknesses**	**Opportunities**	**Threats**
Most comprehensive features set	Resource - Intensive	Potential Partnerships	Resistance to change from other social media
Flexible group/channels system	Limited Accessibility of user accountsfrom university	Feedback and improvement in real time	Competing Solutions
Freandly UI	No anonymity features	University community chat	Connect all university in one app
University multiple content type	Privacy content	Ditch the teacher for better learning	Motivate people to use our app

1.4 Target Group

The target audience includes two different groups:

Students: Students are the primary target group. They are involved in following aspects:Undergraduate and graduate students seeking academic support. Students looking for study groups and project collaborators. Those wanting to share experiences about courses and professors. Students seeking real-time feedback and answers to academic questions

Potential Students: High school graduates researching university programs. Transfer students evaluating the university environment. International students seeking authentic insights into campus life. Individuals exploring specific course experiences and requirements

1.5 Existing Solutions

The landscape of student-focused social platforms has evolved significantly in recent years, driven by changing communication preferences and the increasing digitization of academic communities. This comprehensive analysis examines four distinct platforms that serve various aspects of student social interaction and community building. The platforms under consideration include the berkslv social media application, GoIn Connect's pre-enrollment community platform, Jodel's anonymous hyperlocal network, and the proposed student forum solution designed with Reddit-like functionality.

Each platform addresses unique market segments within the broader student community ecosystem.

The berkslv application focuses on verified Turkish university students through institutional email verification, creating a trusted regional community. GoIn Connect operates in the pre-enrollment space, facilitating connections between admitted students before they arrive at universities, with a business-to-business model targeting educational institutions. Jodel provides anonymous, location-based social networking within a 10-kilometer radius, appealing to students who prefer privacy in their digital interactions. The proposed student forum aims to combine the familiar Reddit interface with student-specific features including channels, polls, and category-based organization.

The analysis reveals significant opportunities in the market for a platform that can effectively combine the strengths of existing solutions while addressing their limitations. The proposed student forum platform has the potential to occupy a unique position in the market through its comprehensive feature set and focus on long-term academic community building rather than ephemeral social interactions.

1.6 Platform Functionality and Core Purpose

1.6.1 berkslv Social Media Application

The berkslv [4] social media application functions as a closed social network exclusively designed for Turkish university students. The platform operates on the principle of verified academic communities, requiring users to authenticate using official university email addresses ending with .edu.tr domains. This verification system ensures that all participants are legitimate members of Turkish higher education institutions, creating a trusted environment for academic and social discourse.

The application serves as a digital campus commons where Turkish students can share experiences, discuss academic matters, and build connections within their national higher education community. Users can create posts about university life, academic challenges, career opportunities, and social events relevant to the Turkish student experience. The platform facilitates connections between students from different universities across Turkey, enabling knowledge sharing and collaborative opportunities that might not otherwise occur.

The focus on Turkish higher education creates a culturally cohesive community where users share common educational experiences, language, and cultural references. This targeted approach allows for discussions that are highly relevant to the specific challenges and opportunities faced by students within the Turkish academic system, including topics such as YKS exam preparation, university transfer processes, and career prospects within the Turkish job market.

1.6.2 GoIn Connect Platform

GoIn Connect [5] operates as a pre-arrival community platform specifically designed to connect admitted students before they begin their university studies. The platform addresses the critical transition pe-

riod between acceptance and enrollment, when prospective students often experience anxiety about moving to new environments, making friends, and adapting to university life. The application creates cohort-based communities organized around specific university programs and entry periods.

The platform facilitates practical information sharing among incoming students, including housing arrangements, visa processes, travel planning, and academic preparation. Students use the platform to find potential roommates, organize group travel to campus, share tips about local amenities, and coordinate social meetings before official orientation programs begin. The application serves as an informal orientation system that operates independently of official university programs.

GoIn Connect functions as a social safety net for international and domestic students who may feel isolated or uncertain about their upcoming university experience. The platform enables students to build support networks and friendships before arriving on campus, reducing the social and emotional challenges associated with starting university life. Universities partner with GoIn Connect to improve student satisfaction, reduce dropout rates, and enhance the overall student experience from the moment of admission.

1.6.3 Jodel Anonymous Network

Jodel [6] operates as a hyperlocal, anonymous social network that connects users within a 10-kilometer geographic radius without revealing their identities. The platform functions as a digital neighborhood bulletin board where users can share thoughts, ask questions, seek advice, and engage in discussions with others in their immediate vicinity. The complete anonymity removes social barriers and allows for more authentic communication about sensitive or personal topics.

The application serves multiple community functions including local information sharing, event announcements, buy-and-sell transactions, and social commentary about neighborhood or campus happenings. Students particularly use Jodel to discuss university-related topics without fear of academic or social repercussions, share honest opinions about courses and professors, and seek help with academic or personal challenges while maintaining privacy.

The voting system allows the community to self-moderate content, with popular posts gaining visibility while inappropriate or irrelevant content becomes less prominent. This democratic approach to content curation creates communities that reflect the collective interests and values of local user populations. The temporal nature of content, combined with geographic boundaries, creates dynamic local conversations that remain relevant to immediate community concerns.

1.7 Feature Comparison and Functionality Analysis

The functional capabilities of each platform reflect their distinct positioning and target audience requirements. Traditional social media features form the foundation of most platforms, but specialized

functionality differentiates each offering in significant ways.

User authentication mechanisms vary considerably across platforms, reflecting different priorities regarding user verification and privacy. The berkslv application requires university email verification, creating a trusted community environment but potentially excluding users who prefer privacy or lack current university affiliation. GoIn Connect uses invitation-based authentication through university partnerships, ensuring users belong to specific cohorts while maintaining some level of identity verification.

Jodel's completely anonymous approach eliminates traditional authentication requirements, allowing users to participate without revealing personal information. This approach reduces barriers to participation but creates challenges for community management and abuse prevention. The proposed student forum includes standard login functionality but has not specified whether additional verification mechanisms would be implemented.

Content creation capabilities demonstrate the platforms' different approaches to user engagement. The berkslv application supports traditional social media posts with standard commenting functionality, providing familiar interaction patterns for users. GoIn Connect facilitates community-focused content sharing with emphasis on practical information exchange and social connection building among incoming students.

Jodel's content model centers on short, anonymous posts called "jodels" within geographic boundaries, creating spontaneous local conversations. The voting mechanism allows community-driven content curation without requiring user identification. The proposed student forum incorporates more sophisticated content types including traditional posts, threaded comments, and dedicated polling functionality, suggesting a more comprehensive approach to student discourse.

Group and community organization features reveal fundamental differences in platform philosophy. The berkslv application appears to rely on broader community interaction without specific group segmentation features. GoIn Connect creates cohort-based communities organized around university programs and entry periods, facilitating relevant connections among students with shared experiences and timelines.

Jodel's location-based model creates implicit communities within geographic boundaries without formal group structures. Users within the same area automatically share a common posting space, creating organic local communities. The proposed student forum incorporates explicit channel and category systems, allowing users to organize discussions around specific topics, courses, or interests with greater granularity than other platforms provide.

Table 1.2 - Comprehensive Platform Feature Comparison

Feature Category	berkslv App	GoIn Connect	Jodel	Student Forum
Authentication	University email required	Invitation-based	Anonymous only	MFA with corporative email
Content Types	Traditional posts	Community updates	Anonymous jodels	Posts, polls, comments
Interaction Model	Standard comments	Community chat	Anonymous replies	Threaded discussions
Search Functionality	Not implemented	Limited	Location-focused	Comprehensive planned
Voting/Polls	Not available	Not available	Post voting	Dedicated poll system
Geographic Scope	Turkey only	Global partnerships	Multi-country	UTM
Target Users	Turkish students	Pre-enrolled students	Local communities	General students and potential UTM users

Overview

The analysis examines: The Four Platforms:

berkslv's Social Media App - A Turkish university-focused platform with email verification GoIn Connect - A pre-enrollment community platform for university partnerships Jodel - An anonymous, hyperlocal social network popular with students Our Student Forum - The proposed Helper discussion platform

Key Insights: Market Positioning: Your forum sits in a unique space with high functionality but needs stronger differentiation. The current competitive landscape shows:

Jodel dominates anonymous local communication GoIn Connect owns the pre-enrollment market through university partnerships berkslv's app serves the verified regional community need Our forum has the most comprehensive feature set but lacks a clear unique selling proposition

2 SYSTEM DESIGN

2.1 System Requirements

2.1.1 Functional Requirements

The student forum application implements the following core functional capabilities:

User Management

FR-1: User registration with email verification - The system shall allow new users to create accounts by providing email, username, and password credentials, followed by mandatory email verification before account activation. **FR-2**: Secure user authentication - The system shall authenticate users through secure login mechanisms using Supabase authentication services with session-based state management. **FR-3**: Profile management - Users shall be able to view and update their profile information including personal details and forum preferences. **FR-4**: Session management - The system shall maintain user sessions securely with automatic timeout and proper session invalidation upon logout.

Content Management

FR-5: Multi-type post creation - Authenticated users shall be able to create, edit, and delete three types of forum posts: basic text posts, interactive polls with multiple options, and event posts with date/time and location information. **FR-6**: Comment system - Users shall be able to add, edit, and delete comments on forum posts with proper threading and moderation capabilities. **FR-7**: Content search - The system shall provide advanced search functionality to locate posts and comments based on keywords, filters, and content type with real-time suggestions. **FR-8**: Channel management system - The system shall organize posts into channels with approval workflows, channel categories (general, academic, social, announcements), and speciality-specific channels for targeted discussions. **FR-15**: Post interaction system - Users shall be able to upvote and downvote posts and comments to promote quality content and community-driven content curation. **FR-16**: Student verification system - The system shall verify student status through university email verification and student card validation (pending approval process) to distinguish verified students from general users.

Academic Services

FR-17: Course review system - Verified students shall be able to review courses per speciality with ratings and written feedback to help fellow students make informed academic decisions. **FR-18**: Professor search and evaluation - The system shall provide a searchable database of professors with aggregated

ratings, reviews, and course associations for academic planning.

Future Features (Planned)

FR-19: Location-based channels - The system shall support local channels that display posts based on geographic proximity to enhance campus-specific discussions (future implementation). **FR-20**: Personal schedule integration - Users shall be able to import and manage personal calendars with course schedules and event integration (future implementation).

Security and Access Control

FR-9: Role-based access control - The system shall implement three user roles: verified students (university email verification), unverified users (limited access), and admin users, with appropriate permissions for content management and administrative functions. **FR-10**: Input validation and sanitization - All user inputs shall be validated server-side and sanitized to prevent injection attacks and ensure data integrity. **FR-11**: Secure API endpoints [7] - All API routes shall require proper authentication and authorization before processing requests. **FR-12**: Professor and student rating system with leaderboards - Verified students shall be able to rate professors across multiple categories (teaching quality, communication, helpfulness) and rate fellow students in categories (academic help, collaboration, leadership, social), with anonymous review capabilities and leaderboard displays based on ratings. **FR-13**: Content recommendation system - The system shall provide intelligent content recommendations to users based on their academic speciality, interaction history, and channel preferences to enhance content discovery and engagement. **FR-14**: Audit logging - The system shall log user activities and security events for monitoring and compliance purposes.

2.1.2 Non-Functional Requirements

The application addresses critical security and performance requirements essential for a secure educational platform:

Security Requirements

NFR-1: Encryption standards - All data transmission shall be encrypted using TLS 1.3, and sensitive data at rest shall be encrypted using AES-256 encryption algorithms. **NFR-2**: Password security - User passwords shall be hashed using bcrypt with a minimum of 12 salt rounds, and password policies shall enforce strong password requirements. **NFR-3**: Cross-Site Scripting (XSS) prevention - The application shall implement Content Security Policy (CSP) headers and input sanitization to prevent XSS attacks. **NFR-4**: Cross-Site Request Forgery (CSRF) protection - All state-changing operations shall be protected with

anti-CSRF tokens and same-site cookie attributes. **NFR-5**: SQL injection prevention - Database operations shall use parameterized queries through Drizzle ORM [8] to prevent SQL injection vulnerabilities. **NFR-6**: Session security - Sessions shall implement secure cookie attributes (HttpOnly, Secure, SameSite) with automatic timeout after 24 hours of inactivity.

Authentication

Our project provides the user with an extensive range of possibilities for registering on the platform. The available options are:

1. Authentication with email and password
2. Additional MFA enrollment for two-factor authentication
3. Authentication using OAuth 2.0: [9]
    - Google account
    - GitHub account (since we are software engineering students)
    - Azure/Microsoft account

Below we explain how each of these authentication methods works and which security measures are taken to mitigate possible attacks.

General authentication principle

The general principle of authentication is that information about the user's session is stored in cookies. The payload of these cookies is encrypted on the server. [10] Cookies can contain the JWT access token and JWT refresh token in most cases, but additional cookies may also be set during the MFA process or OAuth authentication to keep track of the intermediate state of the user. For example, the user is not considered fully authenticated until they provide the TOTP code from their Authenticator app.

Cookies were chosen due to the following reasons:

- Extensive range of settings - unlike local storage, cookies provide more control through properties such as HttpOnly, Secure, and SameSite.
- Isolation from external scripts - by setting the HttpOnly property to true, we ensure that cookies can only be read on the server side of the application, preventing tampering with JavaScript. However, it is important to note that cookies can still be exploited by CSRF attacks.
- Automatic access on each request - cookies are sent with every request that matches their scope and policy. This makes it easy to verify whether the received cookies are still valid, and if not, to invalidate or update them.

Authentication with email and password

Here we describe how authentication with email works. When the user navigates to the `/register` path, they see a sign-up form where they must provide the following information:

1. Nickname
2. Email
3. Password
4. Profile picture

When the user clicks the *Register* button and the input is valid, they receive two things: an email with a confirmation link and a code verification cookie stored in their browser. This ensures that even if the confirmation link is leaked, it has no value without the cookie.

The confirmation link includes a token hash. When the user follows the link, both the token hash and the code verification cookie are sent to the authentication service, where their validity is checked. Once validated, the JWT access and refresh tokens are returned to the user in the form of encrypted cookies. At this point, the user is considered authenticated, and their identity can be confirmed on each request.

Setting up Multi-Factor Authentication

MFA enrollment

As an additional layer of security, we provide the option to use MFA [11]. Each registered user can navigate to the `/settings/mfa-enroll` page to enroll in MFA authentication. They will see a call-to-action button; upon clicking it, a QR code and its equivalent setup key in plain text will be displayed. The user can scan the QR code with an Authenticator app (e.g., Google Authenticator or Microsoft Authenticator). After adding it, they must enter the newly generated code from the application into the input field below the setup key. If the code is correct, the user is considered enrolled.

MFA verification

When the user enters the correct email and password, they receive:

– A challenge ID - used to identify the specific request to verify an MFA factor.
– A factor ID - identifier of the factor the user set up previously.
– A set of cookies with limited access.

From the code's perspective, this means: "You entered the correct login and password, but you still need to prove your identity." The user will receive a new set of cookies as soon as they provide the correct OTP code in the input field.

We also take into consideration how much time the user spends on the MFA input screen. After 60 seconds, the MFA challenge itself expires. This means that even if the OTP is still valid, the user must

restart the login process, since the issued challenge ID is no longer recognized by the service.

MFA Management

The user can also manage their MFA factors. By navigating to `/settings/mfa-manage`, they will see their MFA factor ID. If they choose to delete it, they can press the delete button, after which they will be signed out. They can then log in again using only their email and password.

Future work on MFA

Usually, managing sensitive information such as MFA factors requires reauthentication. Currently, this is not implemented, but it is on the list of planned improvements.

Performance Requirements

NFR-7: Response time - API endpoints shall respond within 500ms for 95% of requests under normal load conditions. **NFR-8**: Scalability - The system shall support concurrent access by up to 1000 authenticated users without performance degradation. **NFR-9**: Database performance - Database queries shall be optimized with proper indexing to maintain sub-100ms response times for common operations.

Reliability and Availability

NFR-10: System availability - The application shall maintain 99.5% uptime during operational hours with automated failover capabilities. **NFR-11**: Data backup - User data and forum content shall be automatically backed up daily with point-in-time recovery capabilities. **NFR-12**: Error handling - The system shall implement comprehensive error handling that prevents sensitive information disclosure while providing meaningful feedback to users.

Compliance and Standards

NFR-13: Security standards compliance - The application shall adhere to OWASP Top 10 security guidelines and implement security best practices throughout the development lifecycle. **NFR-14**: Data privacy - User data handling shall comply with educational privacy standards and implement proper data retention policies. **NFR-15**: Accessibility - The user interface shall meet WCAG 2.1 Level AA accessibility standards to ensure inclusive access for all users.

2.2 System Architecture Overview

The student forum application follows a modern web architecture pattern utilizing Next.js as the full-stack framework, Supabase for authentication and database services, and Drizzle ORM for type-safe database operations. The system employs a component-based design that separates concerns between the frontend presentation layer, server-side business logic, and data persistence layer.

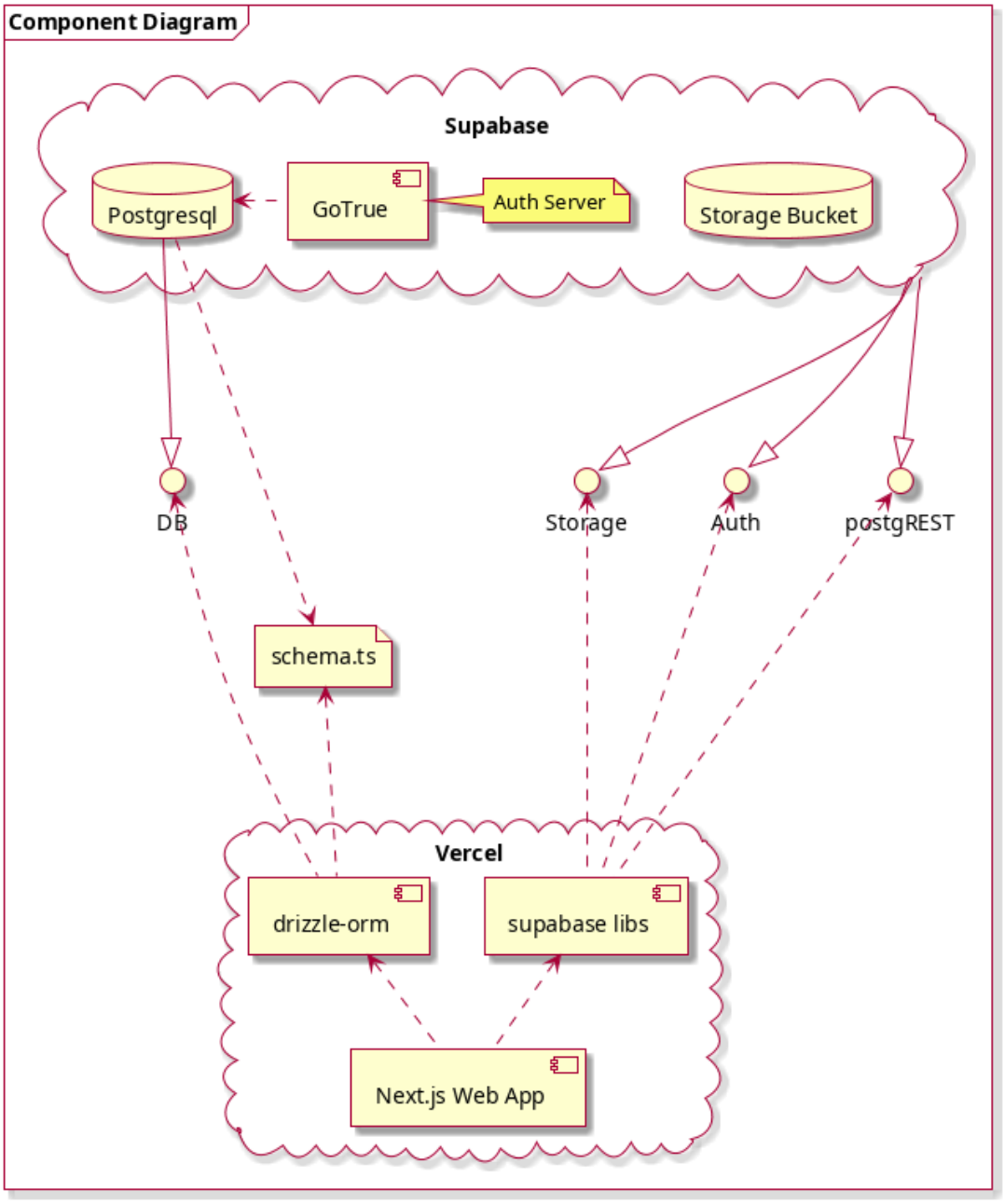


Figure 2.1 - System Component Architecture

The architecture shown in Figure 2.1 demonstrates the interaction between Vercel's hosting environment and Supabase's backend services. The Next.js application utilizes both Drizzle ORM for direct database operations and Supabase libraries for authentication and file storage, creating a hybrid approach that leverages the strengths of each technology.

2.3 Application Feature Design

The Next.js [12] application follows a feature-based architecture where functionality is organized into distinct, modular components that interact through well-defined interfaces. This approach promotes code reusability, maintainability, and separation of concerns.

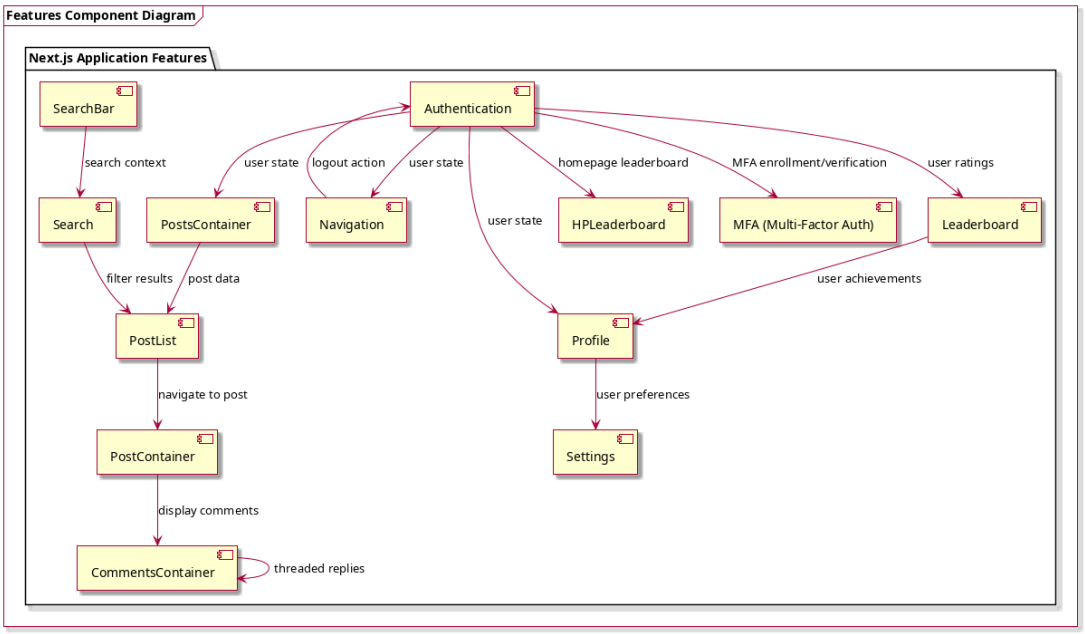


Figure 2.2 - Application Features Component Diagram

The feature architecture shown in Figure 2.2 demonstrates the modular design approach where each feature encapsulates its own components, actions, and types. The Authentication feature serves as the foundation, providing user state management that flows throughout the application. The PostsContainer and related components implement the core forum functionality with proper data flow and dependency management.

Key architectural principles implemented:

- **Separation of Concerns**: Each feature maintains its own actions, components, and types, preventing cross-feature dependencies and promoting modularity.
- **Data Flow Management**: User authentication state flows from the Authentication feature to dependent components, ensuring consistent user context throughout the application.
- **Component Reusability**: Generic UI components are shared across features while maintaining feature-specific business logic within dedicated modules.
- **State Management**: Centralized authentication state with distributed feature-specific state management for optimal performance and maintainability.

2.4 Data Model Design

The application employs a comprehensive relational database schema designed to support the complex relationships inherent in an academic forum platform. The schema encompasses user management, academic structure representation, content organization, and social interaction features while maintaining data integrity and security.

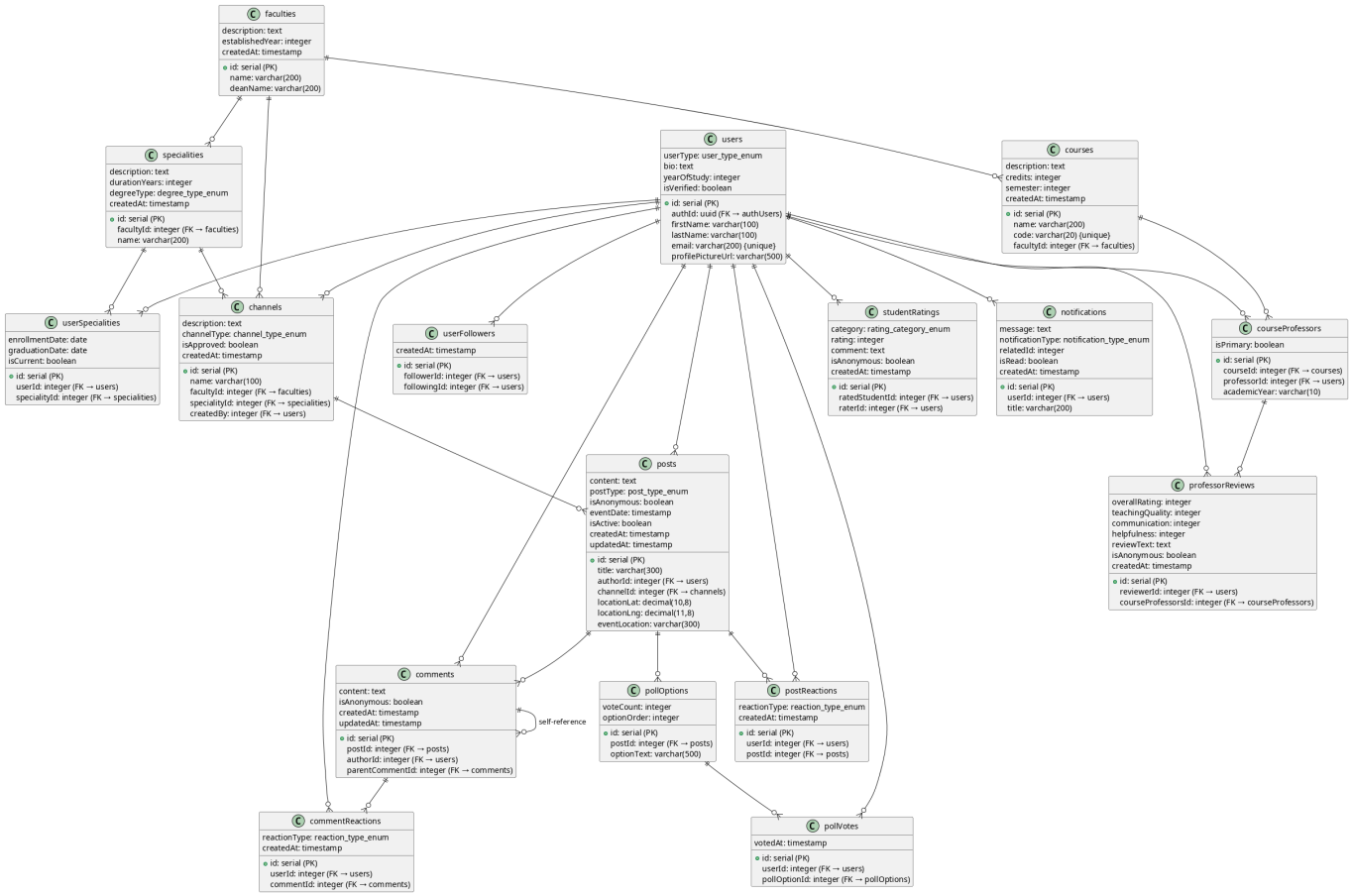


Figure 2.3 - Database Schema Class Diagram

Figure 2.3 illustrates the complete database structure with entities representing users, academic faculties and specialities, forum channels and posts, and various interaction mechanisms. The schema design emphasizes referential integrity through foreign key constraints and implements proper indexing strategies for optimal query performance.

The core entities include:

- **User Management**: Users table with authentication integration, supporting multiple user types (student, professor, admin) with profile information and verification status.
- **Academic Structure**: Faculties, specialities, and courses tables that model the university's organizational hierarchy with proper relationships and constraints.
- **Content System**: Posts, comments, and channels that facilitate forum discussions with support for

anonymous posting, location-based content, and threaded conversations.

– **Interaction Features**: Reactions, ratings, polls, and social following mechanisms that enhance user engagement and content quality assessment.

2.5 Security Workflows

2.5.1 User Login Process

The login workflow implements a server-side action pattern that ensures secure authentication handling. When users submit their credentials, the system processes authentication through Supabase's secure authentication service while managing session cookies at the application level. The middleware layer automatically validates user sessions on subsequent requests, providing seamless protection for authenticated routes.

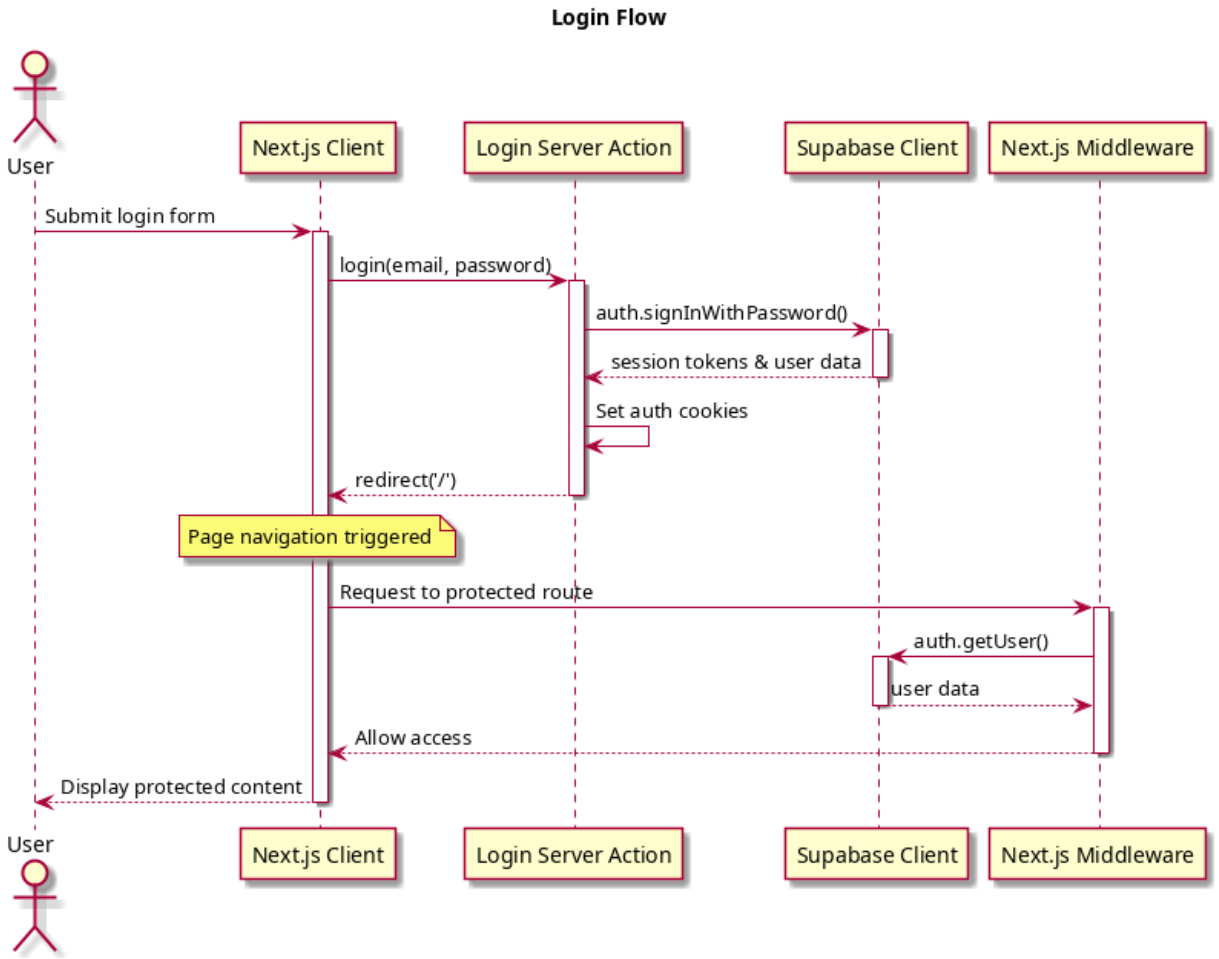


Figure 2.4 - User Login Sequence Flow

Figure 2.4 illustrates the complete login flow, from user credential submission through server-side validation and middleware-based route protection. The process ensures that authentication tokens are securely managed and automatically validated for protected resource access.

2.5.2 User Registration Process

The registration workflow incorporates both user profile creation and email verification processes. The system validates form data, checks for existing users in the database, and creates both authentication records and user profile information atomically. Email confirmation is handled through Supabase's built-in email service, ensuring users verify their accounts before accessing protected features.

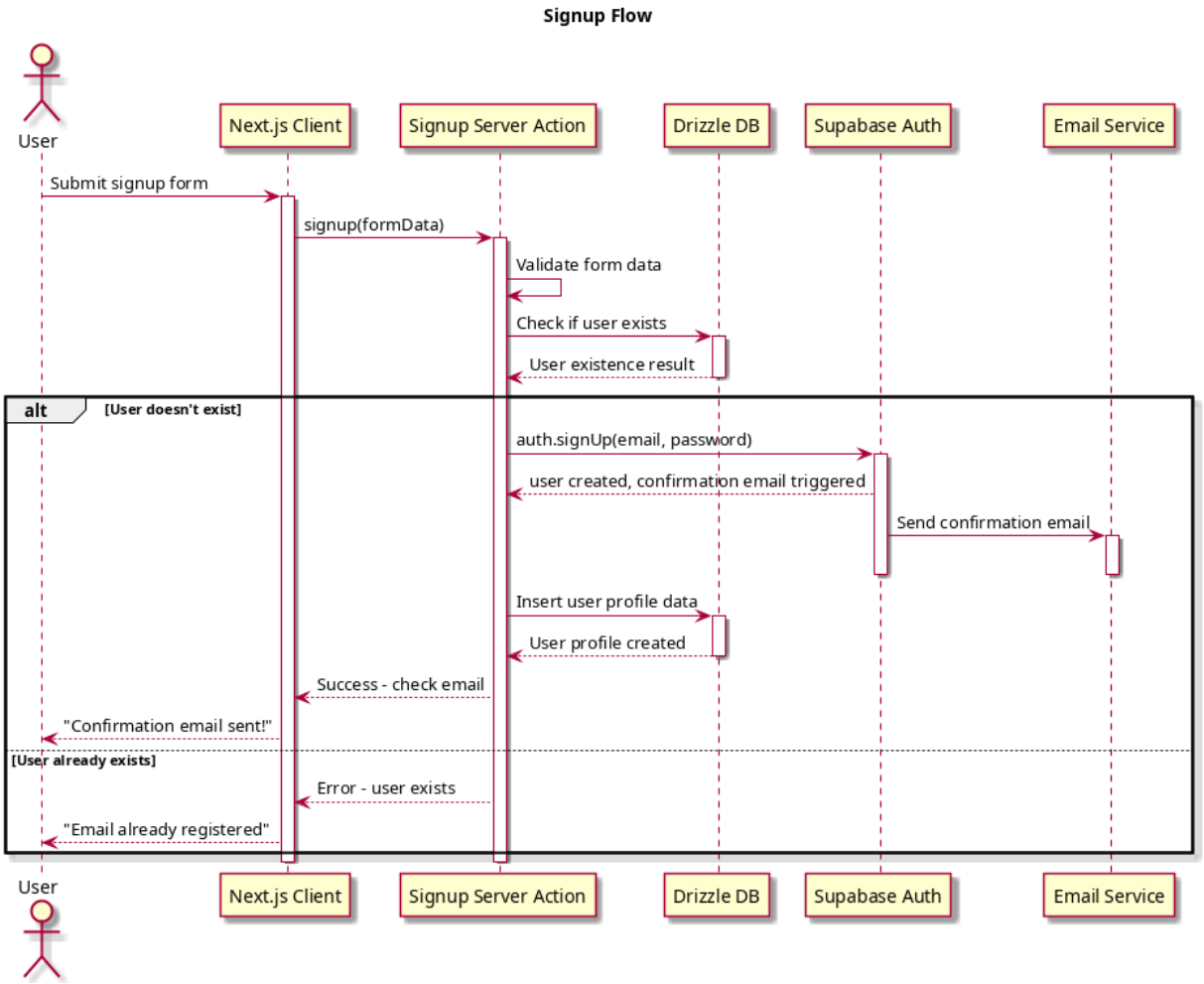


Figure 2.5 - User Registration Sequence Flow

The registration process shown in Figure 2.5 demonstrates the integration between form validation, database operations, and email confirmation services. The system maintains data consistency by performing all operations within a controlled transaction flow.

2.5.3 Email Confirmation Workflow

Email confirmation completes the user verification process through a secure callback mechanism. When users click the confirmation link, the application processes the verification code and establishes a verified user session. This workflow ensures that only users with verified email addresses can access the platform's full functionality.

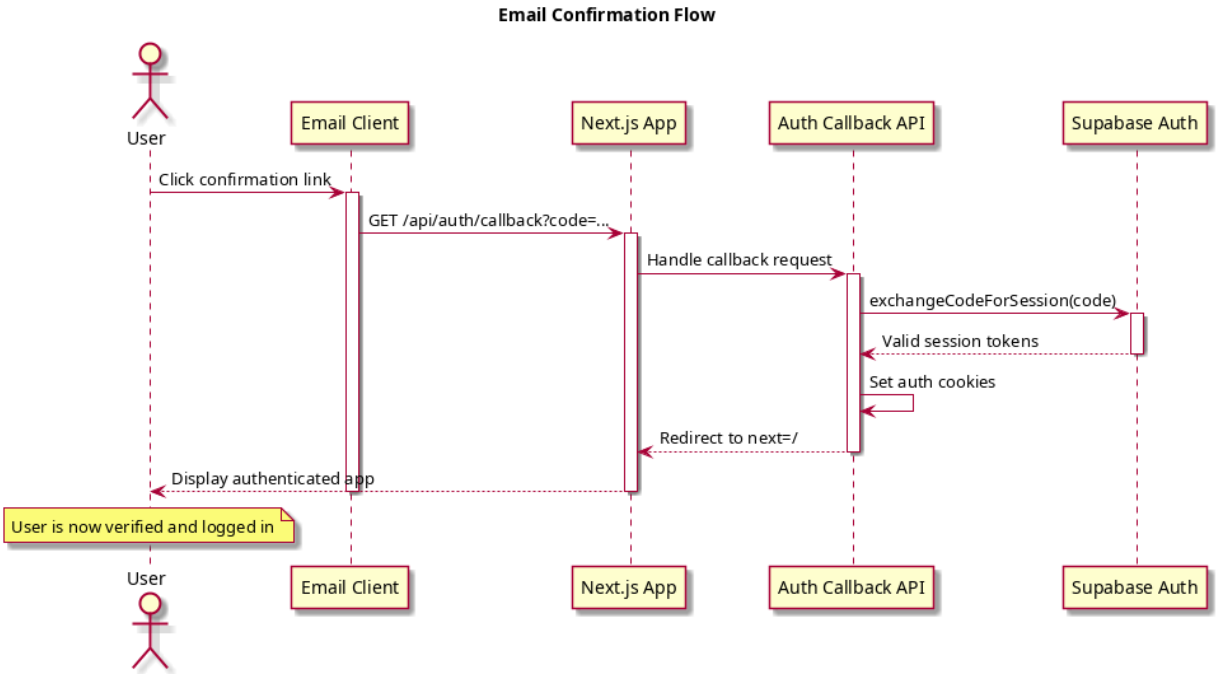


Figure 2.6 - Email Confirmation Sequence Flow

Figure 2.6 shows the secure token exchange process that validates user email addresses and establishes authenticated sessions. The callback API handles the verification seamlessly while maintaining security best practices.

2.5.4 Protected Route Access

The middleware-based protection system automatically validates user authentication status for all protected routes. This approach provides transparent security enforcement without requiring manual authentication checks in individual route handlers. The middleware evaluates user sessions and redirects unauthenticated users to the login page while allowing authenticated users to proceed normally.

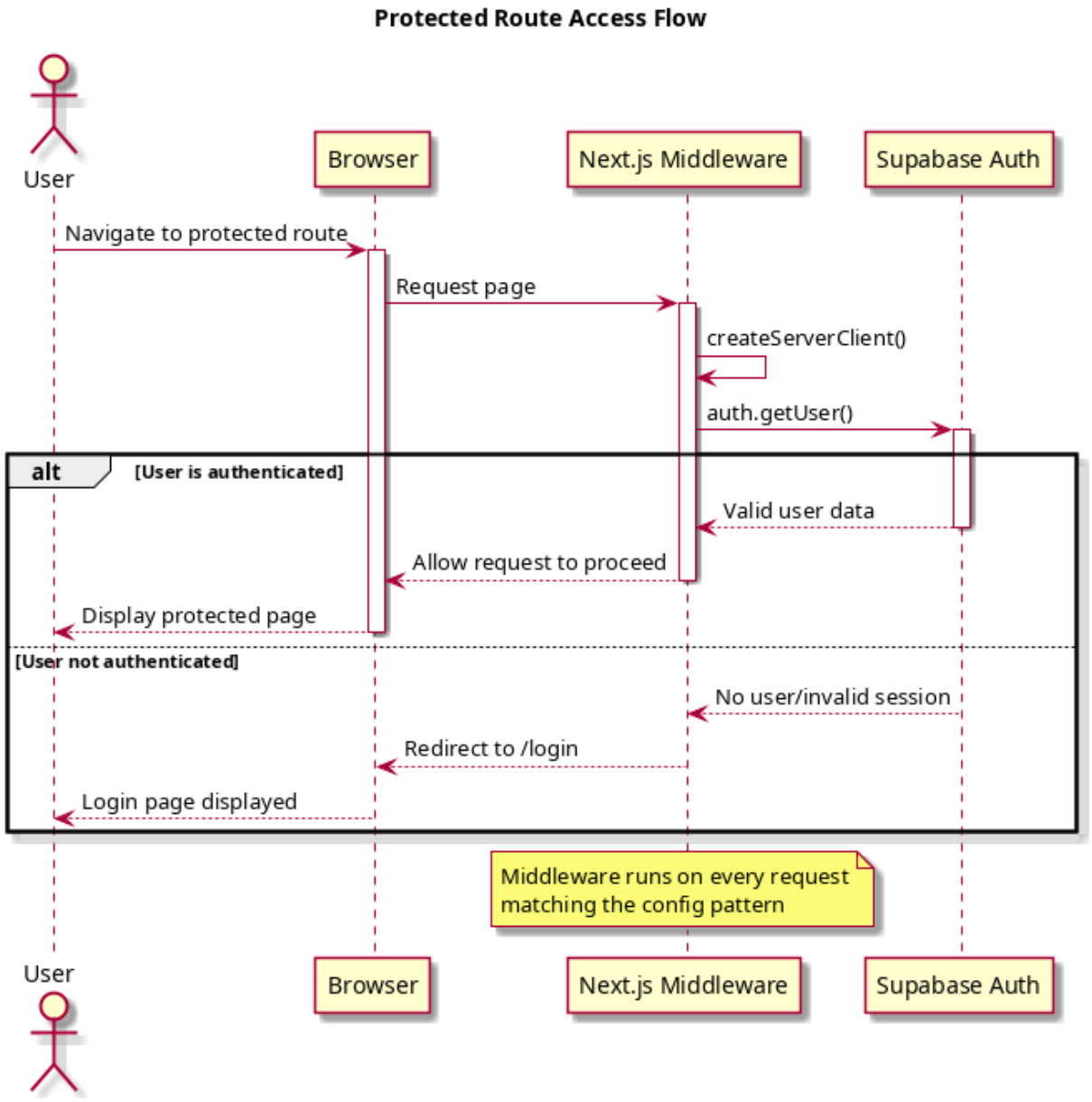


Figure 2.7 - Protected Route Access Sequence Flow

The protection mechanism illustrated in Figure 2.7 ensures consistent security enforcement across the application. The middleware pattern provides a centralized authentication checkpoint that automatically handles both authenticated and unauthenticated access scenarios.

2.6 Security Implementation

The Student Forum platform implements multiple layers of security to protect user data and prevent various attack vectors. This section details the key security mechanisms implemented in the system.

2.6.1 Authentication (Supabase)

Authentication is the process of verifying the identity of a user or system. In the Student Forum application, we use Supabase Auth as the primary authentication provider with server-side session handling implemented via Supabase SSR helpers. This approach ensures that session tokens are handled securely on the server side, preventing client-side manipulation. Session cookies are hardened with `httpOnly: true` to prevent client-side JavaScript from accessing session cookies, mitigating XSS attacks, and `secure: true` in production to ensure cookies are only sent over HTTPS, preventing man-in-the-middle attacks. This is crucial for the application because it protects user accounts from unauthorized access and ensures that only legitimate users can access the forum.

Theoretical Background: Authentication is the first line of defense in any security system. It verifies the identity of users before granting access to protected resources. The most common form of authentication is username/password combination, but modern applications often implement additional security measures such as multi-factor authentication. The principle behind authentication is to establish trust between the user and the system by validating credentials that only the legitimate user should possess.

Example Implementation:

```
// src/utils/supabase/middleware.ts
export async function updateSession(request: NextRequest) {
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { /* cookie options with httpOnly: true */ } }
  );
  await supabase.auth.getUser();
  return NextResponse.next({ request });
}
```

2.6.2 Authorization & Route Protection

Authorization determines what authenticated users are allowed to do within the system. A global Next.js middleware enforces that most pages require a valid Supabase-authenticated user. Unauthenticated requests are redirected to `/login`, except for paths that start with `/login`, `/auth`, or `/error`. Additionally, application-level authorization ensures only verified users can access the system by checking if a user exists in the local `users` table and has `isVerified` set to true. This is essential for the forum application to maintain content integrity and ensure that only registered and verified users can participate in discussions.

Theoretical Background: Authorization is the process of determining what an authenticated user is allowed to do. While authentication verifies who the user is, authorization defines what they can access. The principle of least privilege is fundamental to authorization: users should only have access to the minimum resources necessary to perform their functions. This reduces the potential damage if an account is compromised. Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) are common authorization models.

Example Implementation:

```
// middleware.ts
export async function middleware(request: NextRequest) {
  return await updateSession(request);
}

export const config = {
  matcher: ['/((?!api|_next/static|favicon.ico|auth|login).*)'],
};
```

2.6.3 Multi-Factor Authentication (MFA)

Multi-Factor Authentication (MFA) is a security system that requires more than one method of authentication from independent categories of credentials to verify the user's identity for a login or other transaction. The system includes MFA capabilities with enrollment, verification, and factor management features. During sensitive operations like password updates, the system checks the Supabase Authenticator Assurance Level (AAL) and may require MFA verification. If the required AAL is not met, users are redirected to an MFA challenge step to enhance security for sensitive operations. MFA is vital for the forum application as it adds an extra layer of security beyond passwords, protecting user accounts even if passwords are compromised.

Theoretical Background: MFA is based on the principle of defense in depth, requiring multi-

ple forms of verification from different categories: something you know (password), something you have (phone, token), and something you are (biometrics). Even if one factor is compromised, the attacker still cannot gain access without the other factors. This dramatically increases security and is especially important for sensitive operations like password changes or account modifications.

Example Implementation:

```
// src/app/api/auth/update-password/route.ts
const currentAAL = sessionData?.session?.user?.aal;
if (currentAAL?.level !== 'aal2') {
  return NextResponse.json({
    redirect: '/auth/mfa-challenge',
    message: 'MFA verification required for password change'
  }, { status: 403 });
}
```

2.6.4 Security Headers & CSP

Security headers are HTTP response headers that help protect web applications from various attacks. The Content Security Policy (CSP) is a security standard that helps prevent cross-site scripting (XSS), clickjacking and other code injection attacks. Security headers are set globally via Next.js `headers()` in `next.config.ts`. The CSP includes directives such as `default-src 'self'`, `frame-ancestors 'none'` for clickjacking protection, and `upgrade-insecure-requests`. The policy also specifies allowed sources for various content types while restricting potentially dangerous script sources. Additional headers include `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and `Strict-Transport-Securi` for enhanced security. These headers are critical for the forum application to defend against common web vulnerabilities and protect users from malicious content.

Theoretical Background: Security headers are a defense mechanism that browsers enforce to protect against various client-side attacks. CSP specifically addresses XSS vulnerabilities by defining which sources of content are trusted. X-Frame-Options prevents clickjacking by controlling whether a page can be embedded in a frame. HSTS ensures all communications happen over HTTPS. These headers provide an additional layer of security that works even if other defenses fail.

Example Implementation:

```
// next.config.ts
const nextConfig = {
  async headers() {
```

```
    return [{
      source: '/(.*)',
      headers: [{
        key: 'Content-Security-Policy',
        value: "default-src 'self'; frame-ancestors 'none';"
      }]
    }];
  }
};
```

2.6.5 CSP Violation Reporting

Content Security Policy (CSP) violation reporting allows developers to monitor and analyze potential security issues. The system accepts CSP violation reports through an API endpoint at `/api/csp-report`. This endpoint is rate-limited to prevent abuse and logs violations server-side for security monitoring purposes. The endpoint responds with permissive CORS headers to accommodate browser reporting behavior and returns appropriate status codes. This mechanism is important for the forum application as it helps identify potential security bypasses and allows for continuous improvement of the security policies.

Theoretical Background: CSP violation reporting is part of a security monitoring strategy that allows developers to detect when an attack is attempted or when legitimate content is incorrectly blocked by CSP rules. This provides valuable feedback for tuning security policies and identifying potential security issues before they become serious problems. It's an important component of a defense-in-depth strategy.

Example Implementation:

```
// src/app/api/csp-report/route.ts
export async function POST(request: Request) {
  const report = await request.json();
  console.error('CSP Violation Report:', JSON.stringify(report));
  return new Response(null, { status: 204 });
}
```

2.6.6 Rate Limiting & Abuse Prevention

Rate limiting is a technique used to control the amount of traffic a user or system can consume over a specific period. It prevents abuse, brute force attacks, and resource exhaustion. Rate limiting is implemented using `@upstash/ratelimit` + Upstash Redis to prevent abuse and brute force attacks. Different endpoints

have different rate limits: login attempts are limited to 6 per minute, registration to 4 per minute, post creation to 2 per minute, and comments to 10 per minute. In development and test environments, a no-op rate limiter is used that always allows requests. Rate limiting is essential for the forum application to prevent spam, denial-of-service attacks, and ensure fair resource usage among users.

Theoretical Background: Rate limiting is a crucial defense against automated attacks and resource exhaustion. It helps prevent brute force attacks on authentication endpoints, limits the impact of bots on the system, and ensures fair usage of resources among legitimate users. Rate limiting algorithms like token bucket, leaky bucket, and sliding window counter help control the rate of requests to protect system resources.

Example Implementation:

```
// src/app/api/comments/route.ts
const { success } = await rateLimit.comment.limit(ip);
if (!success) {
  return NextResponse.json(
    { error: 'Too many requests' },
    { status: 429 }
  );
}
```

2.6.7 Input Validation & Sanitization

Input validation and sanitization are security techniques used to ensure that only properly formatted data enters a software system. Input sanitization utilities use DOMPurify to prevent XSS attacks by stripping all HTML tags and attributes, keeping only text content. The system includes validation functions for UUID formats and search queries. Search queries are specially sanitized by trimming, escaping SQL LIKE wildcards (%, _), and removing dangerous characters. These sanitization utilities are applied to user inputs during registration and content creation. This is crucial for the forum application to prevent malicious code injection and maintain data integrity.

Theoretical Background: Input validation and sanitization are fundamental security practices that prevent injection attacks such as XSS, SQL injection, and command injection. The principle is to never trust user input and always validate and sanitize it before processing. Validation ensures data conforms to expected formats, while sanitization removes potentially dangerous content. Both are essential for preventing code injection attacks that could compromise the system.

Example Implementation:

```
// src/lib/security/sanitize.ts
export function sanitize(input: string): string {
  return DOMPurify.sanitize(input, {
    ALLOWED_TAGS: [],  // Strip all tags
    ALLOWED_ATTR: []   // Strip all attributes
  });
}
```

2.6.8 API Error Handling & Safe Responses

Secure error handling prevents information disclosure that could be exploited by attackers. API error handling is implemented to prevent information leakage and maintain security. The system returns structured JSON errors with appropriate HTTP status codes (e.g. `400`, `401`, `404`, `429`, `500`) rather than exposing stack traces to clients. Endpoints validate input early and return appropriate error codes without revealing internal system details. For example, the comments endpoint validates payloads and IDs, returning `400` for invalid input, while the user endpoint returns `401` for authentication issues. This approach prevents attackers from gaining insight into system internals through error messages. Proper error handling is vital for the forum application to avoid exposing sensitive system information that could be used for further attacks.

Theoretical Background: Secure error handling is critical to prevent information disclosure attacks. Detailed error messages can reveal internal system information, database schemas, file paths, and other sensitive data that attackers can use to exploit the system. The principle is to provide enough information for legitimate users to understand what went wrong while revealing nothing about the internal workings of the system to potential attackers.

Example Implementation:

```
// src/app/api/comments/route.ts
if (!postId || typeof postId !== 'string' || !isValidUuid(postId)) {
  return NextResponse.json(
    { error: 'Invalid post ID' },
    { status: 400 }
  );
}
// Return minimal response to prevent information leakage
return NextResponse.json({ success: true }, { status: 201 });
```

2.6.9 Secrets & Environment Configuration

Secure management of sensitive configuration data is critical for application security. The system uses environment variables for configuration, particularly for Supabase credentials. The Supabase configuration is provided via `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` environment variables. These are accessed through the Supabase SSR middleware and are never exposed directly in client-side code. Upstash Redis is loaded via `Redis.fromEnv()` in production-like environments, ensuring that sensitive credentials are properly managed through environment configuration rather than hardcoded values. This approach is essential for the forum application to prevent credential exposure in source code and maintain security across different deployment environments.

Theoretical Background: Secret management is a critical aspect of application security. Hardcoding credentials in source code is a common security mistake that can lead to credential exposure. Best practices include using environment variables, secret management services, or encrypted configuration stores. The principle is to separate secrets from code and ensure they are not stored in version control systems where they could be accidentally exposed.

Example Implementation:

```
// src/utils/supabase/middleware.ts
const supabase = createServerClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
  { cookies: { ... } }
);
```

2.6.10 Open Redirect Protection

Open redirect vulnerabilities occur when an application permits redirection to an arbitrary external domain. The OAuth callback handler validates the `next` query parameter to prevent open redirect attacks. It ensures that the redirect URL is a relative path starting with '/' before performing the redirect. If an absolute URL is detected, the system resets to the homepage to prevent unauthorized redirects to external sites. This protection is important for the forum application to prevent attackers from using the application as a phishing vector.

Theoretical Background: Open redirect vulnerabilities can be exploited for phishing attacks. Attackers craft URLs that appear to come from a trusted site but redirect users to malicious sites. The solution is to validate redirect URLs and only allow redirects to trusted domains or relative paths. This prevents the application from being used as part of a phishing scheme.

Example Implementation:

```
// src/app/api/auth/callback/route.ts
if (next && !next.startsWith('/')) {
  console.warn(`Invalid redirect URL: ${next}`);
  return NextResponse.redirect(`${requestUrl.origin}/`);
}
return NextResponse.redirect(`${requestUrl.origin}${next}`);
```

These security implementations ensure that the Student Forum platform maintains a robust security posture, protecting both user data and system integrity from various attack vectors. The layered approach to security, combining authentication, authorization, input validation, rate limiting, and secure configuration, provides comprehensive protection against common web application threats and ensures a safe environment for academic discussions and feedback.

QUALITY ASSURANCE AND DEPLOYMENT PIPELINE

2.7 Overview of Testing Strategy

The Student Forum application employs a comprehensive testing strategy to ensure code quality, reliability, and security. The testing approach encompasses multiple levels of testing including unit tests, integration tests, and end-to-end tests. This multi-layered testing strategy ensures that individual components function correctly, that they integrate properly with other components, and that the complete user workflows operate as expected.

2.7.1 Testing Framework

The project utilizes Vitest as the primary testing framework. Vitest is a fast and lightweight testing framework that is compatible with Jest's API, making it an excellent choice for Next.js applications. The testing environment is configured with jsdom to simulate a browser environment for DOM-related tests. The configuration is defined in `vitest.config.ts` and includes settings for global test utilities and module aliasing to match the application's import structure.

Configuration Example:

```
import { defineConfig } from 'vitest/config';
import path from 'path';

export default defineConfig({
  test: {
    environment: 'jsdom',
    globals: true,
  },
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
});
```

2.8 Unit Testing

Unit tests form the foundation of the testing strategy, focusing on individual functions, components, and modules in isolation. Each unit test verifies that a specific piece of functionality works as expected under various conditions.

2.8.1 Security Functions Testing

The security functions are thoroughly tested to ensure they properly protect against common vulnerabilities. For example, the sanitization functions are tested to verify they effectively prevent XSS attacks:

Example Test Case:

```
describe('sanitize', () => {
  it('should strip script tags', () => {
    expect(sanitize('<script>alert(1)</script>')).toBe('');
    expect(sanitize('hello<script>alert(1)</script>world')).toBe(
      'helloworld'
    );
  });

  it('should strip HTML tags', () => {
    expect(sanitize('<div>hello</div>')).toBe('hello');
    expect(sanitize('<img src="x" onerror="alert(1)">')).toBe('');
  });

  it('should handle XSS in img tags', () => {
    const xss = '<img src=x onerror="alert(1)">';
    expect(sanitize(xss)).not.toContain('onerror');
    expect(sanitize(xss)).not.toContain('alert');
  });
});
```

What this test does: This test suite verifies that the `sanitize` function properly removes potentially dangerous HTML tags and attributes that could be used for cross-site scripting (XSS) attacks. The first test checks that script tags are completely removed from the input, preventing JavaScript execution. The second test ensures that all HTML tags are stripped, leaving only plain text. The third test specifically targets image tags with event handlers, which are a common vector for XSS attacks. These tests are critical for security

as they ensure that user-generated content cannot inject malicious scripts into the application.

2.8.2 Validation Functions Testing

Validation functions are tested to ensure they correctly identify valid and invalid inputs. For example, the UUID validation function is tested with various inputs:

Example Test Case:

```
describe('isValidUuid', () => {
  it('should return true for valid UUIDs', () => {
    expect(isValidUuid('550e8400-e29b-41d4-a716-446655440000')).toBe(true);
    expect(isValidUuid('6ba7b810-9dad-11d1-80b4-00c04fd430c8')).toBe(true);
  });

  it('should return false for invalid UUIDs', () => {
    expect(isValidUuid('not-a-uuid')).toBe(false);
    expect(isValidUuid('550e8400-e29b-41d4-a716')).toBe(false);
  });
});
```

What this test does: This test suite validates that the `isValidUuid` function correctly identifies properly formatted UUIDs according to the RFC 4122 specification. The first test verifies that valid UUIDs in the correct format (8-4-4-4-12 hexadecimal characters) return true. The second test ensures that malformed UUIDs, non-UUID strings, or incomplete UUIDs return false. This validation is crucial for security and data integrity, as it prevents invalid identifiers from being processed by the system, which could lead to errors or security vulnerabilities.

2.8.3 Search Sanitization Testing

The search sanitization function is tested to ensure it properly handles SQL injection attempts:

Example Test Case:

```
describe('sanitizeSearch', () => {
  it('should escape SQL LIKE wildcards', () => {
    expect(sanitizeSearch('100%')).toBe('100\\%');
    expect(sanitizeSearch('test_value')).toBe('test\\_value');
    expect(sanitizeSearch('path\\to\\file')).toBe('path\\\\to\\\\file');
  });
```

```
  it('should remove dangerous characters', () => {
    expect(sanitizeSearch("test'injection")).toBe('testinjection');
    expect(sanitizeSearch('test"injection')).toBe('testinjection');
  });
});
```

What this test does: This test suite ensures that the `sanitizeSearch` function properly escapes SQL LIKE wildcards (%, _) and removes potentially dangerous characters that could be used for SQL injection attacks. The first test verifies that percent signs and underscores (used in SQL LIKE clauses) are properly escaped with backslashes, preventing wildcard matching in database queries. The second test ensures that quote characters (both single and double quotes) are removed, which prevents SQL injection through string termination and concatenation. This is critical for database security, as it prevents malicious users from crafting search queries that could manipulate SQL statements.

2.9 Integration Testing

Integration tests verify that different modules or services work together correctly. These tests ensure that the interfaces between components function as expected and that data flows properly between different parts of the system.

2.9.1 Authentication Actions Testing

Authentication actions are tested to ensure they properly handle various scenarios including successful logins, failed logins, and MFA requirements:

Example Test Case:

```
describe('login', () => {
  it('return false success if user cannot be signed in', async () => {
    const mockSupabase = {
      auth: {
        signInWithPassword: vi.fn().mockResolvedValue({
          error: { message: 'error' },
          user: null,
        }),
        mfa: { getAuthenticatorAssuranceLevel: vi.fn() },
      },
```

```
    };

    vi.mocked(createClient).mockResolvedValue(mockSupabase as SupabaseClient);

    expect(await login(formState, formData)).toEqual({
      success: false,
      message: 'error',
    });
  });

  it('redirect the valid user to home page if aal level is 1 (no MFA)', async () => {
    const mockSupabase = {
      auth: {
        signInWithPassword: vi.fn().mockResolvedValue({
          error: null,
          user: { id: 'user-1' },
        }),
        mfa: {
          getAuthenticatorAssuranceLevel: vi.fn().mockResolvedValue({
            data: { nextLevel: 'aal1' },
            error: null,
          }),
        },
      },
    };

    vi.mocked(createClient).mockResolvedValue(mockSupabase as SupabaseClient);

    await expect(login(formState, formData)).rejects.toThrow('NEXT_REDIRECT');
    expect(redirect).toBeCalledWith('/');
  });
});
```

What this test does: These tests verify the authentication flow in different scenarios. The first

test simulates a failed login attempt where the Supabase authentication service returns an error. The test ensures that the login function properly handles this error condition by returning a failure state with an appropriate error message. The second test verifies the successful login scenario where the user has an AAL1 (Authentication Assurance Level 1) which means no MFA is required. The test confirms that the user is redirected to the home page ('/') after successful authentication. These tests are essential for security as they ensure that authentication failures are handled gracefully and that successful authentications follow the correct flow.

2.9.2 API Route Testing

API routes are tested to ensure they properly handle requests, validate inputs, and return appropriate responses:

Example Test Case:

```
// Example structure for API route testing
describe('API routes', () => {
  it('should handle valid requests correctly', async () => {
    // Mock request and response objects
    const request = new Request('http://localhost/api/example', {
      method: 'POST',
      body: JSON.stringify({ data: 'test' })
    });

    const response = await handler(request);
    expect(response.status).toBe(200);
    expect(await response.json()).toHaveProperty('success', true);
  });

  it('should return appropriate error for invalid inputs', async () => {
    // Test with invalid data
    const request = new Request('http://localhost/api/example', {
      method: 'POST',
      body: JSON.stringify({ invalid: 'data' })
    });
```

```
    const response = await handler(request);
    expect(response.status).toBe(400);
  });
});
```

What this test does: These tests verify that API routes handle both valid and invalid requests appropriately. The first test sends a valid request with proper data and verifies that the API returns a successful response (HTTP 200) with the expected structure. The second test sends invalid data to ensure that the API properly validates inputs and returns an appropriate error response (HTTP 400 Bad Request). These tests are crucial for API security and reliability, ensuring that the application handles both expected and unexpected inputs gracefully.

2.10 Test Organization and Structure

The test suite is well-organized with tests located alongside the code they test or in dedicated test directories. Test files follow naming conventions such as `*.test.ts` or `*.spec.ts` to clearly identify test files.

2.10.1 Test Categories

The project includes tests for various components:

- **Security utilities**: Testing sanitization and validation functions to ensure they protect against XSS, SQL injection, and other vulnerabilities
- **Authentication**: Testing login, signup, and MFA flows to ensure secure user authentication and proper error handling
- **API routes**: Testing API endpoints and their responses to ensure proper request handling and security
- **Feature actions**: Testing business logic in feature-specific actions to ensure correct functionality
- **Utility functions**: Testing helper functions and utilities to ensure they behave as expected

2.10.2 Mocking Strategy

The testing strategy extensively uses mocking to isolate units of code under test. Dependencies such as Supabase clients, navigation functions, and rate limiters are mocked to ensure tests are predictable and fast.

Example Mocking:

```
vi.mock('@/utils/supabase/server', () => ({
```

```
  createClient: vi.fn(),
}));

vi.mock('next/navigation', () => ({
  redirect: vi.fn(),
}));
```

What this mocking does: The mocking strategy allows tests to isolate the functionality being tested by replacing external dependencies with controlled mock objects. The Supabase client mock allows authentication tests to simulate different authentication outcomes without making actual API calls. The navigation mock allows testing of redirect behavior without actually navigating. This approach ensures tests are fast, deterministic, and focused on the specific functionality being tested.

2.11 Test Coverage and Quality

The testing approach emphasizes comprehensive coverage of edge cases, error conditions, and security considerations. Tests are designed to verify both positive and negative scenarios to ensure the application behaves correctly under various conditions.

2.11.1 Edge Cases Testing

Tests include verification of edge cases such as null inputs, malformed data, and boundary conditions:

Example Edge Case Test:

```
describe('edge cases', () => {
  it('should return empty string for null/undefined', () => {
    expect(sanitize(null as unknown as string)).toBe('');
    expect(sanitize(undefined as unknown as string)).toBe('');
  });

  it('should preserve whitespace in text', () => {
    expect(sanitize('  hello  ')).toBe('  hello  ');
  });
});
```

What this test does: This test suite handles edge cases that could cause runtime errors or unexpected behavior. The first test ensures that the sanitize function gracefully handles null or undefined inputs by

returning an empty string instead of crashing. This prevents potential runtime errors if the function receives unexpected input types. The second test verifies that the function preserves legitimate whitespace in text, ensuring that normal text formatting is maintained while still removing potentially dangerous content. These tests are important for application stability and user experience.

2.11.2 Security Testing

Special attention is given to security-related tests that verify protection against common vulnerabilities:

- Cross-site scripting (XSS) prevention: Tests ensure that user input is properly sanitized to prevent malicious script injection
- SQL injection protection: Tests verify that user input is properly escaped to prevent database manipulation
- Input validation and sanitization: Tests confirm that all user inputs are validated and sanitized before processing
- Authentication and authorization controls: Tests verify that only authorized users can access protected resources

2.11.3 Security Analysis Tools

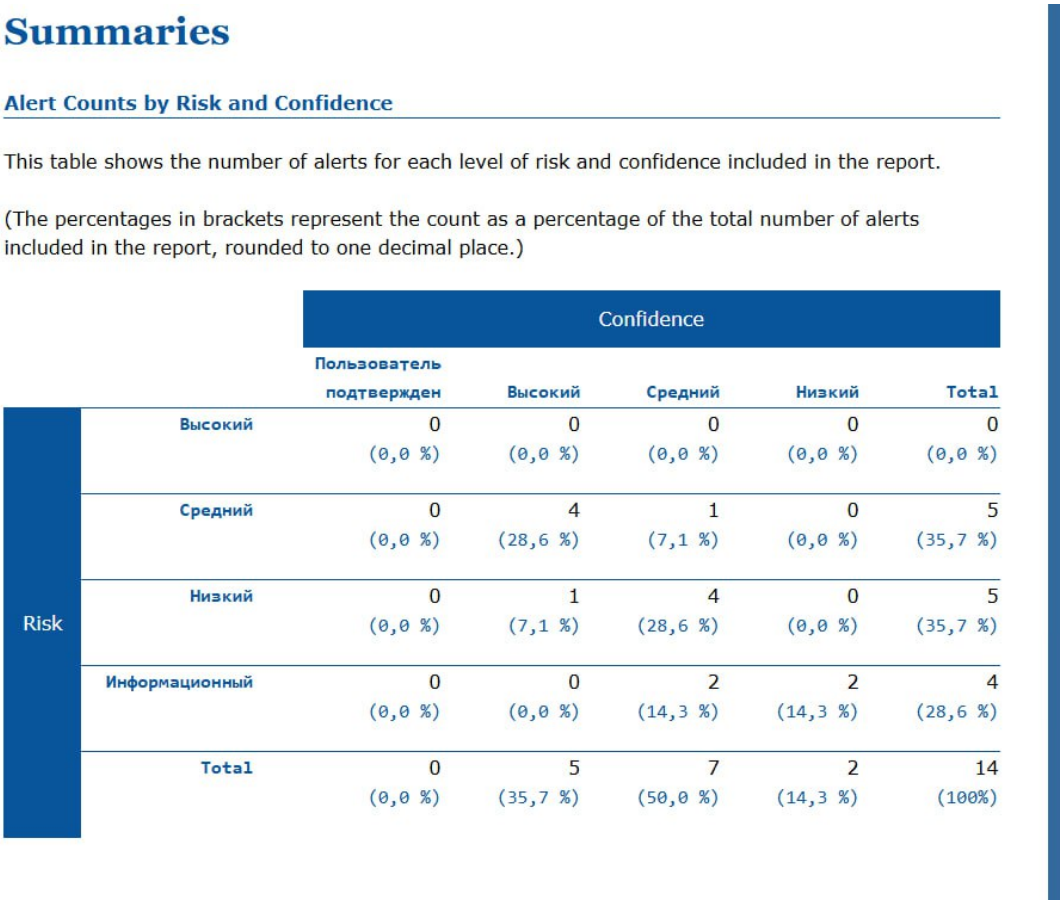
In addition to the comprehensive testing strategy, the project employs automated security analysis tools to identify and address potential vulnerabilities:

GitHub Security Analyzer: GitHub Security Analyzer was configured and used to scan all pull requests targeting the dev branch. During the analysis, it was discovered that the custom text sanitization solution was not fully reliable and could potentially allow security issues. To address this, the decision was made to replace the custom sanitization logic with DOMPurify, a well-established and secure library for sanitizing user-generated content. After applying this change and fixing the reported vulnerabilities, GitHub Security Analyzer confirmed that no critical security issues remained in the project.

Dependabot: Dependabot was configured to continuously monitor the project's dependencies for known security vulnerabilities. The bot automatically scans the project's dependencies and creates pull requests to update vulnerable packages to secure versions. This proactive approach ensures that security patches are applied promptly, reducing the window of exposure to known vulnerabilities. Dependabot also helps maintain the project's security posture by regularly checking for outdated dependencies that may contain unpatched security flaws.

OWASP ZAP Security Scanning: OWASP ZAP was re-run to reassess the overall security status of the application. The scan confirmed that there were no serious or critical vulnerabilities present. Issues

related to Content Security Policy (CSP) were resolved, and the remaining minor warnings were evaluated and determined to be non-critical and not harmful to the application's security. The OWASP ZAP scanning process is illustrated in the following figure:



Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		Пользователь подтвержден	Высокий	Средний	Низкий	Total
Risk	Высокий	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)	0 (0,0 %)
	Средний	0 (0,0 %)	4 (28,6 %)	1 (7,1 %)	0 (0,0 %)	5 (35,7 %)
	Низкий	0 (0,0 %)	1 (7,1 %)	4 (28,6 %)	0 (0,0 %)	5 (35,7 %)
	Информационный	0 (0,0 %)	0 (0,0 %)	2 (14,3 %)	2 (14,3 %)	4 (28,6 %)
	Total	0 (0,0 %)	5 (35,7 %)	7 (50,0 %)	2 (14,3 %)	14 (100%)

Figure 2.8 - OWASP ZAP Security Scanning Interface

2.12 Logging and Monitoring

2.12.1 Theory of Logging

Logging is a critical aspect of software development that involves recording events, activities, and states within an application. Logs serve multiple purposes including debugging, monitoring system health, detecting security incidents, and providing insights into application behavior. Effective logging strategies help developers and system administrators understand how an application performs in production environments, identify issues quickly, and maintain system reliability.

Key principles of effective logging include:

- **Log Levels**: Differentiating between debug, info, warning, error, and critical messages to categorize the severity of events
- **Structured Logging**: Using consistent formats that include timestamps, log levels, and contextual information to facilitate analysis

– **Performance Considerations**: Ensuring logging does not significantly impact application performance
– **Security**: Avoiding logging sensitive information such as passwords, tokens, or personal data
– **Retention Policies**: Managing log storage and archival to balance historical data needs with storage costs

2.12.2 Sentry as Logging Solution

For the Student Forum application, we implemented Sentry as our primary error tracking and monitoring solution. Sentry is a popular open-source platform that provides real-time error tracking, performance monitoring, and debugging tools for various programming languages and frameworks. It enables developers to capture, aggregate, and analyze errors occurring in production environments, allowing for rapid identification and resolution of issues.

Sentry offers several advantages for our application:

– **Real-time Error Tracking**: Automatically captures and reports errors as they occur in the application
– **Detailed Error Context**: Provides stack traces, user information, and environmental details to help diagnose issues
– **Performance Monitoring**: Tracks application performance metrics and identifies slow operations
– **Release Health**: Monitors the stability of different application releases
– **Alerting System**: Sends notifications when new errors occur or when error rates exceed predefined thresholds

The following figures show the Sentry dashboard interface and an example of captured error logs:

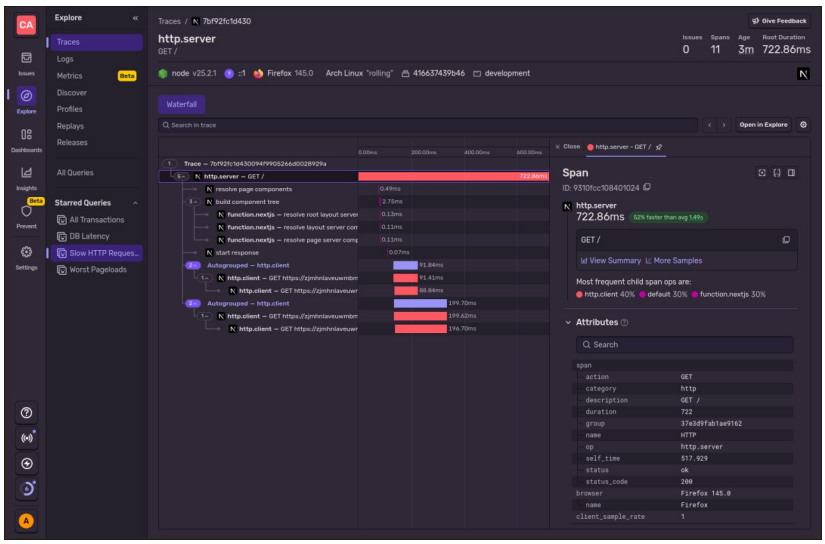


Figure 2.9 - Sentry Dashboard Interface

Figure 2.9 illustrates the Sentry dashboard interface, which provides a comprehensive overview of application errors and performance metrics. The dashboard displays key statistics including the number of new issues, event counts, and project activity. It features a clean, intuitive interface that allows developers to quickly identify and prioritize critical issues. The left sidebar contains navigation options for accessing different aspects of error tracking, release health, and performance monitoring. The main panel presents a chronological list of errors, with each issue showing its severity level, title, and the last occurrence timestamp. This centralized view enables development teams to efficiently monitor application health and respond to problems in real-time.

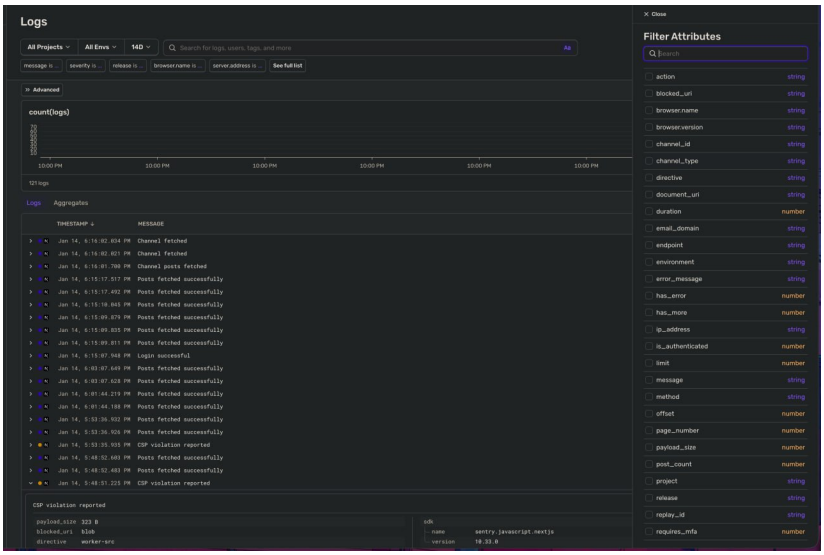


Figure 2.10 - Sentry Error Log Example

Figure 2.10 displays a detailed view of a specific error captured by Sentry. This interface provides comprehensive information about individual errors, including the complete stack trace, contextual data about the user session, device information, and environmental variables. The error details panel shows the exact location where the error occurred in the code, along with variable values at the time of the exception. Additional tabs provide access to breadcrumbs (preceding events leading to the error), user information, and custom tags that help with categorization and filtering. This granular level of detail enables developers to quickly diagnose and resolve issues by understanding the complete context in which errors occur.

This comprehensive testing approach ensures that the Student Forum application maintains high quality, security, and reliability across all components and user interactions. The tests provide confidence that the application handles both normal and exceptional conditions appropriately, protecting both users and the system from potential vulnerabilities.

2.13 Continuous Integration and Deployment (CI/CD)

2.13.1 CI/CD Pipeline Overview

The Student Forum application implements a robust Continuous Integration and Continuous Deployment (CI/CD) pipeline to automate the process of building, testing, and deploying code changes. The CI/CD pipeline ensures that every code commit undergoes rigorous validation before being deployed to production, maintaining the application's stability and security.

The main deployment pipeline consists of two parallel jobs that must both pass before deployment:

- **Job 1**: Unit tests and linting checks
- **Job 2**: End-to-end tests with Playwright

Once both jobs pass successfully, the project is automatically deployed to Vercel. For non-master branches, the deployment occurs in preview mode, while for the master branch, it deploys in production mode.

2.13.2 Main Deployment Workflow

The project utilizes GitHub Actions as the primary CI/CD platform. The main workflow is configured with two parallel jobs that run simultaneously. The first job handles unit tests and linting checks, while the second job runs end-to-end tests with Playwright. The deployment job only executes if both test jobs pass successfully. For the master branch, the application is deployed to Vercel in production mode, while for other branches it's deployed in preview mode.

2.13.3 Additional Security Runners

The project includes additional security-focused runners that complement the main CI/CD pipeline:

CodeQL Scanning for GitHub Actions

CodeQL scanning is configured to verify the security of CI/CD files themselves. This runner analyzes the GitHub Actions workflow files to identify potential security vulnerabilities in the automation scripts.

CodeQL for Source Code

CodeQL scanning is also configured for the application source code to verify security in pull requests. Additionally, a cron job runs a full security scan on the entire codebase once per week to ensure ongoing security compliance.

This CodeQL workflow provides continuous security analysis of the codebase. It runs on every push

and pull request to main, and additionally performs a comprehensive weekly scan of the entire codebase. The workflow analyzes both JavaScript and TypeScript code to identify potential security vulnerabilities, ensuring that security is maintained throughout the development lifecycle.

2.13.4 Deployment Strategies

The application employs several deployment strategies to ensure zero-downtime deployments and quick rollbacks if needed:

- **Vercel Preview Deployments**: For non-master branches, Vercel automatically creates preview deployments with unique URLs for testing
- **Production Deployments**: For the master branch, Vercel deploys to the production environment with automatic SSL certificates and CDN distribution
- **Rollback Mechanisms**: Vercel provides instant rollback capabilities to previous deployments if issues arise

2.13.5 Deployment Status Checks

As part of the CI/CD pipeline, GitHub displays a deployment check status when code is pushed to the repository. This provides immediate visual feedback on the status of the automated checks and deployment process. The following figure shows the deployment check status that appears on the GitHub interface when code is pushed:

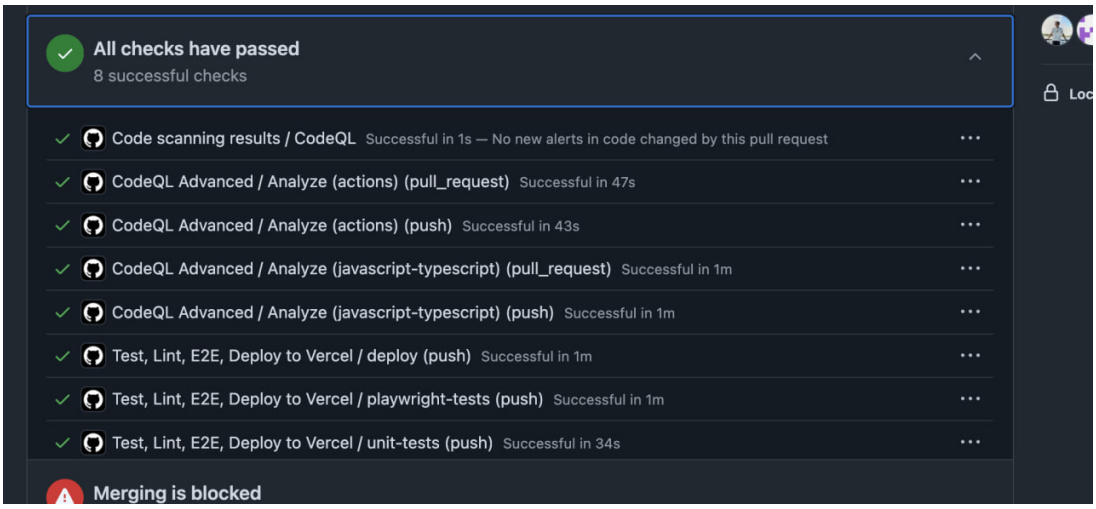


Figure 2.11 - GitHub Deployment Check Status

Figure 2.11 shows the deployment check status that appears on the GitHub interface when code is pushed. This visual indicator provides immediate feedback on the status of the automated checks, including the parallel unit tests, E2E tests, linting, and security scans. The deployment check ensures that all required validations pass before allowing the merge or deployment to proceed, maintaining the quality and security

standards of the Student Forum application.

2.13.6 Benefits of the CI/CD Implementation

The implemented CI/CD pipeline provides several key benefits:

- **Parallel Testing**: Running unit tests and E2E tests in parallel reduces total pipeline execution time
- **Comprehensive Validation**: Both code quality (linting) and functionality (unit and E2E tests) are verified before deployment
- **Automated Security Scanning**: Multiple layers of security checks protect against vulnerabilities
- **Preview Environments**: Automatic preview deployments for non-master branches enable easy testing and review
- **Zero-Downtime Deployments**: Vercel's deployment infrastructure ensures no service interruption during updates
- **Consistent Environments**: Ensures that staging and production environments are identical
- **Visual Status Indicators**: Clear deployment check status on GitHub provides immediate feedback on pipeline health

This comprehensive CI/CD implementation ensures that the Student Forum application can be reliably and securely updated with new features and security patches while maintaining high availability and performance for users.

CONCLUSIONS

In summary, the evidence presented supports the idea that a student-focused forum is a good idea for implementation, as it allows students to centralize and give them a convenient way to connect with each other and discuss topics of concern to them.

It's also worth noting that this platform requires a high level of security, as it uses students' personal information and their anonymous posts and comments. Various security measures were employed to ensure user safety. Specifically, the OAuth protocol was implemented, multi-factor authentication (MFA) support was implemented, and the Supabase platform, with its encryption and access control support, was used for data storage and processing.

On the frontend, Next.js was used to implement protection mechanisms against common attacks (XSS, CSRF), and secure cookie attributes were used. All API requests are validated, preventing the possibility of SQL injections and other vulnerabilities.

As a result, the project demonstrates the practical application of Secure Application Development principles, combining forum functionality with modern security measures, ensuring resilience to cyberthreats and meeting the stated requirements.

In addition to providing security and essential functionality, the project establishes the groundwork for future growth and expansion. Other features that might be added to the forum include role-based moderating tools, interaction with university services, and sophisticated analytics to track user behavior and community involvement. This demonstrates the project's usefulness as it has the potential to develop into a strong platform that fosters peer connections as well as academic cooperation and knowledge exchange inside the university.

BIBLIOGRAPHY

1. AGHA, Halima and HASHMI, Kiran. “Social Media Applications and their Effect on Undergraduate Students’ Learning in Higher Education”. In: *Voyage Journal of Educational Studies* 4.3 (2024). Accessed: 2025-10-02, pp. 20–34. DOI: 10.58622/vjes.v4i3.174. URL: https://www.researchgate.net/publication/383419046_Social_Media_Applications_and_their_Effect_on_Undergraduate_Students%27_Learning_in_Higher_Education.

2. DOE, John and SMITH, Jane. “Impact of Social Media on Academic Performance of Students in Higher Education”. In: *International Journal of Science, Engineering and Management (IJSEM)* 11.7 (2023). Accessed: 2025-10-02, pp. 115–123. URL: https://ijsem.org/article/ijsem_vol11_iss7_15.pdf.

3. AUTHOR, A. and COAUTHOR, B. “The Role of a Major Social Media Platform on Students’ Academic Performance: Perception Versus Reality”. In: *Electronic Journal of Intercultural Mediation and Education (EJIMED)* (2023). Accessed: 2025-10-02, pp. 1–10. URL: https://www.ejimed.com/download/the-role-of-a-major-social-media-platform-on-students-academic-performance-perception-versus-reality-14135.pdf.

4. BERKSLV. *Social Media App – Open Source Project* [online]. Accessed: 2025-10-02. 2024. URL: https://github.com/berkslv/social-media-app.

5. GOIN CONNECT. *Goin Connect – Student Networking Platform* [online]. Accessed: 2025-10-02. 2024. URL: https://www.goinconnect.com/.

6. JODEL LABS. *Jodel – Hyperlocal Community App* [online]. Accessed: 2025-10-02. 2024. URL: https://jodel.com/en/.

7. SUPABASE. *Supabase — The Open Source Firebase Alternative* [online]. Accessed: 2025-10-02. 2025. URL: https://supabase.com/.

8. DRIZZLE TEAM. *Drizzle ORM — A Type-Safe SQL ORM for TypeScript* [online]. Accessed: 2025-10-02. 2025. URL: https://orm.drizzle.team/.

9. GOOGLE. *Google Identity — OAuth 2.0 Protocols* [online]. Accessed: 2025-10-02. 2025. URL: https://developers.google.com/identity/protocols/oauth2?hl=ru.

10. SARKAR, Nurul and RASHID, Mamunur. “OAuth 2.0: A Framework to Secure the OAuth-Based Service for Packaged Web Application”. In: *Innovative Perspectives on Interactive Communication Systems and Technologies*. IGI Global, 2020, pp. 92–139. DOI: 10.4018/978-1-7998-3355-

0 . ch005. URL: https : / / www . researchgate . net / publication / 341336409 _ OAuth _ 20 _ A _ Framework_to_Secure_the_OAuth-Based_Service_for_Packaged_Web_Application.

11. MICROSOFT SUPPORT. *What Is Multifactor Authentication?* [online]. Accessed: 2025-10-02. 2025. URL: https://support.microsoft.com/en-gb/topic/what-is-multifactor-authentication-e5e39437-121c-be60-d123-eda06bddf661.

12. VERCEL / NEXT.JS TEAM. *Next.js — The React Framework* [online]. Accessed: 2025-10-02. 2025. URL: https://nextjs.org/.